

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA**



BÁO CÁO ĐỒ ÁN TỐT NGHIỆP

**HỌC TĂNG CƯỜNG ĐỂ TỐI ƯU HÓA
KHẢ NĂNG TƯƠNG TÁC CỦA AI TRONG GAME**

Sinh viên: Văn Thị Minh Huyền

Mã số sinh viên: 22010329

Khóa: K16

Ngành: Khoa học máy tính (Tài năng) **Hệ:** Đại học chính quy

Giảng viên hướng dẫn: Th.S. Nguyễn Văn Sơn

HÀ NỘI, THÁNG 8 NĂM 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA

BÁO CÁO ĐỒ ÁN TỐT NGHIỆP

HỌC TĂNG CƯỜNG ĐỂ TỐI ƯU HÓA
KHẢ NĂNG TƯƠNG TÁC CỦA AI TRONG GAME

Họ và tên: Văn Thị Minh Huyền

Mã sinh viên: 22010329

Khóa: K16

Ngành: Khoa học máy tính (Tài năng) **Hệ:** Đại học chính quy

Giảng viên hướng dẫn: Th.S. Nguyễn Văn Sơn

HÀ NỘI, THÁNG 8 NĂM 2026

LỜI CAM ĐOAN

Tên tôi là: Văn Thị Minh Huyền

Mã sinh viên: 22010329

Ngành: Khoa học máy tính

Tôi đã thực hiện đồ án tốt nghiệp với đề tài: Học tăng cường để tối ưu hóa khả năng tương tác của AI trong game

Tôi xin cam đoan đây là đề tài nghiên cứu của riêng tôi và được sự hướng dẫn của Th.S Nguyễn Văn Sơn.

Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa được các tác giả khác công bố dưới bất kỳ hình thức nào. Nếu phát hiện có bất kỳ hình thức gian lận nào tôi xin hoàn toàn chịu trách nhiệm trước pháp luật.

GIẢNG VIÊN HƯỚNG DẪN

(Ký, ghi rõ họ tên)

Hà Nội, ngày ... tháng ... năm 20...

SINH VIÊN

(Ký, ghi rõ họ tên)

LỜI CẢM ƠN

Lời đầu tiên, em xin bày tỏ lòng biết ơn sâu sắc đến quý thầy cô Trường Công nghệ thông tin Phenikaa đã tận tình giảng dạy, truyền đạt kiến thức và tạo mọi điều kiện thuận lợi cho em trong suốt quá trình học tập tại trường.

Em xin gửi lời cảm ơn chân thành đến thầy Nguyễn Văn Sơn - giảng viên hướng dẫn đồ án tốt nghiệp, đã dành nhiều thời gian, công sức định hướng, góp ý và hỗ trợ em trong từng giai đoạn thực hiện đề tài. Những chỉ dẫn tận tâm của thầy là nền tảng quan trọng giúp em hoàn thành đồ án này.

Cuối cùng, em xin bày tỏ lòng biết ơn sâu sắc đến gia đình - những người đã luôn đồng hành, động viên và tạo điều kiện tốt nhất để em có thể toàn tâm hoàn thành chương trình học.

Do kiến thức và kinh nghiệm còn hạn chế, đồ án không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của quý thầy cô và các bạn để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn!

TÓM TẮT ĐỒ ÁN

Đồ án nghiên cứu việc ứng dụng học tăng cường (Reinforcement Learning — RL) để tối ưu hóa khả năng tương tác của các tác nhân trí tuệ nhân tạo (AI agent) trong môi trường game ba chiều. Trọng tâm của đề tài là xây dựng ba môi trường game 3D đặc trưng trên nền tảng Unity ML-Agents — *Football Table* (đối kháng một-một), môi trường *Cooperative Puzzle* (hợp tác đa tác nhân) và *Cross The Road* (điều hướng tránh phương tiện) — rồi tiến hành huấn luyện và so sánh có hệ thống ba thuật toán tiêu biểu: PPO (Proximal Policy Optimization), SAC (Soft Actor-Critic) và MA-POCA (Multi-Agent POsthumous Credit Assignment).

Mỗi thuật toán được huấn luyện trên từng môi trường với cùng điều kiện phần cứng và được đánh giá theo bốn tiêu chí định lượng: tốc độ hội tụ, tỷ lệ hoàn thành nhiệm vụ, tổng phần thưởng trung bình và độ ổn định huấn luyện. Đề tài cũng trình bày quy trình tích hợp thuật toán off-policy (SAC) ra ngoài trainer tích hợp sẵn của ML-Agents thông qua Python Low-Level API, nhằm khắc phục giới hạn của ML-Agents khi SAC không hỗ trợ cơ chế self-play và group reward.

Kết quả thực nghiệm cho thấy không tồn tại một thuật toán tối ưu duy nhất cho mọi bài toán: PPO kết hợp self-play phù hợp nhất với môi trường đối kháng, SAC đạt hiệu quả mẫu (sample efficiency) vượt trội trong bài toán điều hướng đơn tác nhân, còn MA-POCA thể hiện ưu thế rõ rệt trong bài toán hợp tác đa tác nhân nhờ cơ chế centralized critic. Những kết luận này cung cấp cơ sở tham khảo cho việc lựa chọn thuật toán RL phù hợp với từng cơ chế game trong thực tiễn phát triển.

Từ khóa: Học tăng cường, Deep Reinforcement Learning, Unity ML-Agents, PPO, SAC, MA-POCA, AI trong game, đa tác nhân.

Mục lục

DANH MỤC TỪ VIẾT TẮT	ix
1 MỞ ĐẦU	1
1.1 Lý do chọn đề tài	1
1.2 Mục tiêu nghiên cứu	2
1.3 Đối tượng và phạm vi nghiên cứu	3
1.4 Tổng quan tài liệu và khoảng trống nghiên cứu	3
1.5 Phương pháp nghiên cứu	4
1.6 Ý nghĩa khoa học và thực tiễn	5
1.7 Bố cục đồ án	6
2 CƠ SỞ LÝ THUYẾT	6
2.1 Trí tuệ nhân tạo trong game	6
2.1.1 AI truyền thống	7
2.1.2 AI học máy	7
2.2 Nền tảng Reinforcement Learning	8
2.2.1 Các khái niệm cơ bản	8
2.2.2 Markov Decision Process (MDP)	10
2.2.3 Policy và Value Function	10
2.3 Học tăng cường sâu	11
2.3.1 Vai trò của mạng Neural Network trong RL	12
2.3.2 Phân loại: Policy-Based, Value-Based, Actor-Critic	12
2.4 Các thuật toán sử dụng trong đề tài	13
2.4.1 PPO (Proximal Policy Optimization)	14
2.4.2 SAC (Soft Actor-Critic)	15
2.4.3 MA-POCA (Multi-Agent POsthumous Credit Assignment)	16
2.5 Unity ML-Agents Toolkit	18
2.5.1 Kiến trúc tổng thể	18
2.5.2 Quy trình huấn luyện Agent	19
2.5.3 Hỗ trợ thuật toán và giới hạn tích hợp sẵn	20
3 THIẾT KẾ MÔI TRƯỜNG VÀ PHƯƠNG PHÁP THỰC NGHIỆM	21

3.1	Tổng quan thiết kế hệ thống	21
3.1.1	Kiến trúc huấn luyện tổng thể	21
3.1.2	Không gian quan sát	22
3.1.3	Không gian hành động	22
3.1.4	Nguyên tắc thiết kế hàm phần thưởng	23
3.2	Môi trường 1: Football Table	24
3.2.1	Mô tả bài toán và môi trường	24
3.2.2	Thiết kế tác nhân và luật chơi	24
3.2.3	Hàm phần thưởng	25
3.2.4	Lựa chọn và cấu hình thuật toán	25
3.3	Môi trường 2: Cooperative Puzzle (hợp tác đa tác nhân)	26
3.3.1	Mô tả bài toán và môi trường	26
3.3.2	Thiết kế tác nhân và cơ chế phối hợp	26
3.3.3	Hàm phần thưởng	27
3.3.4	Lựa chọn và cấu hình thuật toán	27
3.4	Môi trường 3: Vượt đường tránh phương tiện	28
3.4.1	Mô tả bài toán và môi trường	28
3.4.2	Thiết kế tác nhân và chương ngại vật	28
3.4.3	Hàm phần thưởng	29
3.4.4	Lựa chọn và cấu hình thuật toán	29
3.5	Bảng tổng hợp đặc trưng ba môi trường	30
3.6	Triển khai thuật toán ngoài ML-Agents	30
3.6.1	Vấn đề với SAC trong ML-Agents	30
3.6.2	Cách tích hợp thuật toán tùy chỉnh qua Python API	31
3.6.3	Cấu trúc mã nguồn bổ sung	32
3.7	Thiết lập thực nghiệm	32
3.7.1	Cấu hình phần cứng và phần mềm	32
3.7.2	Tham số huấn luyện từng thuật toán	33
3.7.3	Tiêu chí đánh giá	34
4	KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	34
4.1	Kết quả môi trường Football Table	35
4.2	Kết quả môi trường vượt đường (Cross The Road)	37

4.3	Kết quả môi trường Cooperative Puzzle	40
4.4	Đánh giá tổng hợp	42
4.4.1	So sánh tốc độ hội tụ	42
4.4.2	So sánh tỷ lệ thành công	43
4.4.3	So sánh độ ổn định	43
4.4.4	Bảng chứng từ chỉ số huấn luyện nội tại	44
4.4.5	Phân tích ưu nhược điểm từng thuật toán	44
4.5	Thảo luận	45
4.5.1	Khả năng ứng dụng trong phát triển game	45
4.5.2	Hạn chế của mô hình và các vấn đề còn tồn tại	46
KẾT LUẬN		46
PHỤ LỤC A. Cấu hình huấn luyện PPO		52
PHỤ LỤC B. Cấu hình huấn luyện SAC		52
PHỤ LỤC C. Cấu hình huấn luyện MA-POCA		53
PHỤ LỤC D. Mã nguồn minh họa		54
PHỤ LỤC E. Kết quả TensorBoard		55

Danh sách hình vẽ

2.1	Vòng lặp tương tác giữa agent và môi trường trong học tăng cường	9
2.2	Kiến trúc tổng thể của Unity ML-Agents Toolkit	19
4.1	Tiến trình chỉ số xếp hạng ELO của PPO tự chơi trên Football Table	35
4.2	Đường cong phần thưởng đánh giá trên môi trường Football Table .	36
4.3	Đường cong phần thưởng thực đo trên môi trường Cross The Road	37
4.4	Độ dài lượt chơi thực đo trên môi trường Cross The Road	38
4.5	Diễn biến entropy của chính sách trên môi trường Cross The Road .	39
4.6	Hàm mất mát của mạng phê bình trên môi trường Cross The Road .	40
4.7	Đường cong phần thưởng trên môi trường Cooperative Puzzle . . .	40
4.8	Chỉ số nội tại của MA-POCA trên môi trường Cooperative Puzzle: entropy (trục trái) và phần thưởng tò mò (trục phải)	42
4.9	So sánh tỷ lệ thành công của ba thuật toán trên ba môi trường . . .	43

Danh sách bảng

2.1	Phân loại các thuật toán sử dụng trong đề tài	13
3.1	Hàm phần thưởng của môi trường Football Table	25
3.2	Hàm phần thưởng của môi trường Cooperative Puzzle	27
3.3	Hàm phần thưởng của môi trường Cross The Road	29
3.4	Tổng hợp đặc trưng ba môi trường thực nghiệm	30
3.5	Cấu hình phần cứng và phần mềm thực nghiệm	33
3.6	Tóm tắt siêu tham số huấn luyện chính	33
4.1	Kết quả ba thuật toán trên môi trường Football Table	36
4.2	Kết quả ba thuật toán trên môi trường Cross The Road (thực đo) . .	37
4.3	Kết quả ba thuật toán trên môi trường Cooperative Puzzle	41
4.4	Chỉ số huấn luyện nội tại thực đo (các môi trường hành động rời rạc)	44
4.5	Xếp hạng thuật toán theo từng môi trường	45
4.6	Tóm tắt chỉ số cuối thực đo từ TensorBoard	55

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Diễn giải
AI	Artificial Intelligence	Trí tuệ nhân tạo
RL	Reinforcement Learning	Học tăng cường
Deep RL	Deep Reinforcement Learning	Học tăng cường sâu
MDP	Markov Decision Process	Quá trình quyết định Markov
PPO	Proximal Policy Optimization	Thuật toán tối ưu chính sách vùng lân cận
SAC	Soft Actor-Critic	Thuật toán actor-critic cực đại hóa entropy
MA-POCA	Multi-Agent POsthumous Credit Assignment	Thuật toán gán công lao hậu định cho đa tác nhân
TRPO	Trust Region Policy Optimization	Tối ưu chính sách theo vùng tin cậy
DQN	Deep Q-Network	Mạng Q sâu
GAE	Generalized Advantage Estimation	Ước lượng advantage tổng quát
FSM	Finite State Machine	Máy trạng thái hữu hạn
GOAP	Goal-Oriented Action Planning	Lập kế hoạch hành động hướng mục tiêu
NPC	Non-Player Character	Nhân vật phi người chơi
MLP	Multi-Layer Perceptron	Mạng perceptron nhiều lớp

Từ viết tắt	Tiếng Anh	Diễn giải
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
ELO	Elo Rating System	Hệ thống xếp hạng Elo
API	Application Programming Interface	Giao diện lập trình ứng dụng
gRPC	google Remote Procedure Call	Giao thức gọi thủ tục từ xa của Google
YAML	YAML Ain't Markup Language	Định dạng tệp cấu hình
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
SB3	Stable-Baselines3	Thư viện hiện thực các thuật toán RL

1. MỞ ĐẦU

1.1 Lý do chọn đề tài

Trong khoảng một thập kỷ trở lại đây, trí tuệ nhân tạo đã có những bước tiến vượt bậc và len lỏi vào hầu hết các lĩnh vực của đời sống, từ y tế, tài chính, giao thông cho đến giải trí. Ngành công nghiệp game là một trong những lĩnh vực chịu ảnh hưởng sâu sắc nhất, bởi lẽ trò chơi điện tử vừa là môi trường thử nghiệm lý tưởng cho các thuật toán học máy, vừa là sản phẩm thương mại có nhu cầu rất lớn về những nhân vật máy tính thông minh và có khả năng tương tác tự nhiên với người chơi. Trong bức tranh đó, học tăng cường nổi lên như một hướng tiếp cận đặc biệt phù hợp: nó cho phép một tác nhân tự học cách hành động thông qua quá trình thử và sai, mà không cần đến một tập dữ liệu được gán nhãn sẵn.

Khả năng của học tăng cường đã được chứng minh qua hàng loạt thành tựu mang tính bước ngoặt. Một hệ thống học tăng cường có thể chơi thành thạo hàng chục trò chơi điện tử cổ điển chỉ từ hình ảnh thô của màn hình [11], có thể đánh bại kỳ thủ cờ vây hàng đầu thế giới [17], hay phối hợp nhiều tác nhân để chiến thắng trong những trò chơi chiến thuật phức tạp [12, 20]. Những kết quả này cho thấy học tăng cường không chỉ có giá trị học thuật mà còn mở ra tiềm năng ứng dụng thực tiễn rất lớn trong việc xây dựng hành vi cho nhân vật máy tính.

Tuy nhiên, một trong những rào cản lớn nhất khi nghiên cứu và ứng dụng học tăng cường chính là việc xây dựng môi trường huấn luyện phù hợp. Các môi trường hai chiều đơn giản hoặc các bộ dữ liệu chuẩn thường được dùng trong nghiên cứu học thuật tuy thuận tiện nhưng lại không phản ánh được độ phức tạp của trò chơi thực tế. Một môi trường ba chiều với mô phỏng vật lý chân thực, không gian quan sát phong phú và các tình huống thay đổi liên tục mới thực sự thử thách khả năng học của tác nhân, đồng thời gắn với bài toán phát triển game thương mại hơn. Việc thiếu vắng những môi trường như vậy, cùng với chi phí xây dựng chúng, khiến cho việc so sánh các thuật toán học tăng cường trong bối cảnh game ba chiều vẫn còn là một khoảng trống đáng chú ý.

Nền tảng Unity kết hợp với bộ công cụ ML-Agents cung cấp một hệ sinh thái đủ mạnh để giải quyết vấn đề trên. Unity là một trong những công cụ phát triển game phổ biến nhất hiện nay, hỗ trợ mô phỏng vật lý và dựng hình ba chiều ở

mức chuyên nghiệp; trong khi ML-Agents đóng vai trò cầu nối, cho phép kết nối trực tiếp môi trường game với các thuật toán học tăng cường hiện đại được cài đặt trong hệ sinh thái Python. Sự kết hợp này biến Unity thành một nền tảng lý tưởng để thiết kế các môi trường huấn luyện đa dạng, từ bài toán đối kháng, hợp tác cho đến điều hướng tránh vật cản trong không gian ba chiều.

Xuất phát từ những lý do trên, tôi lựa chọn đề tài “*Học tăng cường để tối ưu hóa khả năng tương tác của AI trong game*” với trọng tâm là xây dựng ba môi trường game ba chiều mang tính chất khác nhau trên nền tảng Unity ML-Agents, sau đó huấn luyện và so sánh một cách có hệ thống ba thuật toán tiêu biểu là PPO, SAC và MA-POCA trên từng loại môi trường. Kết quả của đề tài không chỉ làm rõ điều kiện phù hợp của từng thuật toán mà còn cung cấp một bộ môi trường có thể tái sử dụng cho các nghiên cứu và dự án phát triển game về sau.

1.2 Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề tài là nghiên cứu, xây dựng và đánh giá quy trình ứng dụng học tăng cường để huấn luyện tác nhân trí tuệ nhân tạo trong môi trường game ba chiều trên nền tảng Unity ML-Agents. Từ mục tiêu tổng quát đó, đề tài hướng đến bốn mục tiêu cụ thể.

Thứ nhất, đề tài xây dựng ba môi trường game ba chiều có tính chất khác biệt để đại diện cho ba dạng bài toán học tăng cường phổ biến. Môi trường thứ nhất là *Football Table*, mô phỏng một bàn bi lắc ba chiều, đại diện cho bài toán đối kháng một đối một với không gian hành động liên tục. Môi trường thứ hai là môi trường *Cooperative Puzzle*, trong đó nhiều tác nhân phải phối hợp với nhau để cùng hoàn thành một mục tiêu chung. Môi trường thứ ba là *Cross The Road*, trong đó tác nhân phải điều hướng vượt qua dòng phương tiện di chuyển liên tục để đến đích.

Thứ hai, đề tài thiết kế hệ thống quan sát, hành động và phần thưởng phù hợp với đặc thù của từng môi trường, sao cho tác nhân có đủ thông tin cần thiết để học được hành vi hiệu quả trong không gian ba chiều mà không rơi vào những hành vi bất thường ngoài mong muốn.

Thứ ba, đề tài triển khai huấn luyện ba thuật toán PPO, SAC và MA-POCA trên từng môi trường trong cùng điều kiện phân cứng, đồng thời thu thập và phân tích kết quả theo bốn tiêu chí định lượng là tốc độ hội tụ, tỷ lệ hoàn thành nhiệm vụ.

vụ, tổng phần thưởng trung bình và độ ổn định của quá trình huấn luyện.

Thứ tư, trên cơ sở các kết quả thu được, đề tài so sánh và phân tích ưu nhược điểm của từng thuật toán ứng với từng loại bài toán, từ đó đưa ra khuyến nghị lựa chọn thuật toán phù hợp cho các cơ chế game khác nhau trong thực tiễn phát triển.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài gồm ba nhóm chính. Nhóm thứ nhất là ba môi trường game ba chiều được xây dựng trên nền tảng Unity, bao gồm môi trường đối kháng Football Table, môi trường Cooperative Puzzle đa tác nhân và môi trường điều hướng Cross The Road. Nhóm thứ hai là ba thuật toán học tăng cường được khảo sát, gồm PPO (thuật toán tối ưu chính sách vùng lân cận), SAC (thuật toán actor-critic cực đại hóa entropy) và MA-POCA (thuật toán gán công lao cho đa tác nhân). Nhóm thứ ba là bản thân nền tảng Unity ML-Agents cùng giao diện lập trình cấp thấp bằng Python phục vụ việc tích hợp thuật toán tùy chỉnh.

Về phạm vi nội dung, đề tài tập trung vào thiết kế môi trường ba chiều, xây dựng hệ thống quan sát, hành động, phần thưởng, huấn luyện tác nhân và đánh giá hiệu năng thuật toán. Đề tài không đi sâu vào các khía cạnh tối ưu hóa đồ họa, âm thanh hay hoàn thiện sản phẩm game thương mại. Ba môi trường được lựa chọn nhằm bao phủ ba dạng bài toán học tăng cường điển hình là đối kháng một đối một, hợp tác đa tác nhân và điều hướng đơn tác nhân.

Về phạm vi thuật toán, ba thuật toán PPO, SAC và MA-POCA được chọn dựa trên mức độ phổ biến trong nghiên cứu, tính sẵn có trong hệ sinh thái ML-Agents và sự đa dạng về mặt kiến trúc, khi chúng đại diện cho ba hướng tiếp cận khác nhau lần lượt là on-policy, off-policy và đa tác nhân. Toàn bộ thực nghiệm được tiến hành trên một cấu hình máy tính cố định để bảo đảm tính công bằng khi so sánh, chi tiết cấu hình này được trình bày trong Mục 3.7.1.

1.4 Tổng quan tài liệu và khoảng trống nghiên cứu

Nghiên cứu về học tăng cường trong game đã trải qua nhiều giai đoạn phát triển. Mnih và cộng sự [11] lần đầu chứng minh rằng học tăng cường sâu có thể đạt trình độ con người trong các trò chơi điện tử cổ điển chỉ từ dữ liệu điểm ảnh thô, mở ra hướng ứng dụng học tăng cường trực tiếp vào môi trường game. Tiếp nối thành

tự đó, Silver và cộng sự [17, 18] đạt bước đột phá với các hệ thống chơi cờ vây ở trình độ kiện tướng thế giới, khi kết hợp học tăng cường với mạng nơ-ron sâu và tìm kiếm cây.

Ở quy mô lớn hơn với môi trường phức tạp và nhiều tác nhân, các hệ thống chơi trò chơi chiến thuật thời gian thực [12, 20] đã cho thấy học tăng cường có thể xử lý những bài toán với không gian trạng thái khổng lồ, nhiều tác nhân hoạt động đồng thời và đòi hỏi chiến lược dài hạn. Về phía thuật toán, Schulman và cộng sự [16] đề xuất PPO như một phương pháp on-policy vừa ổn định vừa dễ triển khai, trong khi Haarnoja và cộng sự [4] giới thiệu SAC với cơ chế điều hòa entropy giúp cân bằng giữa khám phá và khai thác, đặc biệt hiệu quả với không gian hành động liên tục. Trong lĩnh vực đa tác nhân, Cohen và cộng sự [2] phát triển MA-POCA với cơ chế phê bình tập trung nhằm giải quyết bài toán gán công lao trong môi trường hợp tác.

Về mặt công cụ, Juliani và cộng sự [6] giới thiệu Unity ML-Agents như một hệ sinh thái mã nguồn mở tích hợp sẵn khả năng xây dựng môi trường ba chiều với mô phỏng vật lý chân thực, hỗ trợ huấn luyện tác nhân trực tiếp và được sử dụng rộng rãi trong cả nghiên cứu lẫn phát triển game.

Mặc dù đã có nhiều công trình nghiên cứu về từng thuật toán riêng lẻ trên các môi trường chuẩn, vẫn còn thiếu những nghiên cứu tập trung vào việc xây dựng môi trường game ba chiều tùy chỉnh làm nền tảng huấn luyện và so sánh đồng thời nhiều thuật toán. Cụ thể, phần lớn các môi trường huấn luyện hiện có vẫn là hai chiều hoặc ở dạng vector đơn giản, chưa khai thác được tiềm năng của môi trường ba chiều với vật lý mô phỏng trong việc tạo ra các tình huống học phức tạp và thực tế hơn. Bên cạnh đó, cũng chưa có nhiều nghiên cứu đặt ba thuật toán PPO, SAC và MA-POCA lên cùng một bàn cân trên một bộ môi trường đa dạng về tính chất, từ đối kháng, hợp tác cho đến điều hướng. Đề tài này hướng đến việc lấp đầy khoảng trống đó bằng cách xây dựng ba môi trường game ba chiều đặc trưng và tiến hành một bộ thực nghiệm so sánh có hệ thống.

1.5 Phương pháp nghiên cứu

Đề tài sử dụng kết hợp ba nhóm phương pháp nghiên cứu bổ trợ lẫn nhau. Phương pháp nghiên cứu lý thuyết được dùng để tổng hợp và phân tích các tài liệu khoa học

về học tăng cường, học tăng cường sâu và ba thuật toán PPO, SAC, MA-POCA, dựa trên các bài báo gốc cùng tài liệu kỹ thuật của Unity ML-Agents; đây là cơ sở cho việc thiết kế môi trường và lựa chọn tham số huấn luyện.

Phương pháp thực nghiệm được sử dụng để xây dựng ba môi trường game ba chiều trong Unity với đầy đủ hệ thống vật lý, quan sát, hành động và phần thưởng, sau đó triển khai và huấn luyện từng thuật toán trên mỗi môi trường trong cùng điều kiện phần cứng và số bước huấn luyện. Trong quá trình này, dữ liệu huấn luyện được thu thập và trực quan hóa theo thời gian thực bằng công cụ TensorBoard.

Cuối cùng, phương pháp so sánh và đánh giá được dùng để đối chiếu hiệu năng của các thuật toán dựa trên bốn tiêu chí định lượng là tốc độ hội tụ, tỷ lệ hoàn thành nhiệm vụ, tổng phần thưởng trung bình và độ ổn định huấn luyện, kết hợp với phân tích định tính về mức độ phù hợp của từng thuật toán với từng dạng môi trường.

1.6 Ý nghĩa khoa học và thực tiễn

Về mặt khoa học, đề tài đề xuất và hiện thực hóa một quy trình xây dựng môi trường game ba chiều phục vụ huấn luyện học tăng cường trên nền tảng Unity, có thể được tái sử dụng như một bộ môi trường tham chiếu mở cho các nghiên cứu tiếp theo. Bên cạnh đó, bộ thực nghiệm so sánh có hệ thống giữa PPO, SAC và MA-POCA trên ba dạng môi trường đặc trưng góp phần làm rõ điều kiện phù hợp của từng thuật toán trong bối cảnh game ba chiều. Ngoài ra, quy trình tích hợp thuật toán off-policy vào Unity thông qua giao diện lập trình cấp thấp bằng Python cũng được trình bày chi tiết, mở ra một hướng đi có thể tái sử dụng cho các nghiên cứu về học off-policy trong môi trường Unity.

Về mặt thực tiễn, ba môi trường ba chiều được xây dựng trong đề tài có thể được sử dụng trực tiếp làm nền tảng thử nghiệm trí tuệ nhân tạo trong các dự án game Unity khác. Kết quả so sánh thuật toán giúp các nhà phát triển có cơ sở lựa chọn thuật toán học tăng cường phù hợp với từng cơ chế game, chẳng hạn cơ chế đối kháng, hợp tác hay điều hướng, mà không phải tiến hành lại toàn bộ thực nghiệm từ đầu. Quy trình thiết kế hàm phần thưởng và không gian quan sát trong môi trường ba chiều được trình bày trong đề tài cũng có giá trị tham khảo cho các nhóm phát triển game có nhu cầu tích hợp trí tuệ nhân tạo.

1.7 Bố cục đồ án

Ngoài phần Mở đầu và Kết luận, nội dung chính của đồ án được tổ chức thành ba chương. Chương 2 trình bày cơ sở lý thuyết, bao gồm tổng quan về trí tuệ nhân tạo trong game, nền tảng lý thuyết của học tăng cường với quá trình quyết định Markov, chính sách và hàm giá trị, các kiến trúc học tăng cường sâu, cùng đặc điểm của ba thuật toán PPO, SAC và MA-POCA; chương này cũng giới thiệu nền tảng Unity ML-Agents và các giới hạn tích hợp liên quan đến việc triển khai thuật toán tùy chỉnh.

Chương 3 trình bày chi tiết quá trình thiết kế ba môi trường game ba chiều trong Unity, bao gồm không gian quan sát, không gian hành động và hàm phần thưởng của từng môi trường; đồng thời mô tả phương pháp tích hợp thuật toán SAC ra ngoài bộ huấn luyện tích hợp sẵn của ML-Agents, thiết lập thực nghiệm và các tiêu chí đánh giá được sử dụng. Cuối cùng, Chương 4 trình bày kết quả huấn luyện của từng thuật toán trên từng môi trường thông qua biểu đồ và bảng số liệu, tiến hành so sánh tổng hợp, phân tích ưu nhược điểm của từng thuật toán theo từng dạng bài toán và thảo luận về khả năng ứng dụng thực tiễn cùng hướng phát triển tiếp theo.

2. CƠ SỞ LÝ THUYẾT

Chương này thiết lập nền tảng lý thuyết cho toàn bộ đồ án. Trước hết, chương điềm lại quá trình phát triển của trí tuệ nhân tạo trong game, từ các kỹ thuật lập trình hành vi truyền thống đến sự trỗi dậy của học máy. Tiếp đó, chương trình bày các khái niệm nền tảng của học tăng cường, bao gồm quá trình quyết định Markov, chính sách và hàm giá trị, rồi mở rộng sang học tăng cường sâu. Phần trọng tâm của chương phân tích nguyên lý hoạt động cùng ưu nhược điểm của ba thuật toán được sử dụng trong đề tài, và cuối cùng giới thiệu nền tảng Unity ML-Agents cùng những giới hạn tích hợp của nó.

2.1 Trí tuệ nhân tạo trong game

Trí tuệ nhân tạo trong game có lịch sử phát triển song hành với sự ra đời của ngành công nghiệp game từ những thập niên 1970–1980. Khác với AI trong nghiên cứu học thuật, AI trong game không nhất thiết phải tối ưu hay thông minh tuyệt đối -

mục tiêu cốt lõi là tạo ra những đối thủ hoặc nhân vật phi người chơi (NPC) có hành vi đủ thuyết phục, góp phần mang lại trải nghiệm thú vị và thử thách cho người chơi [9]. Theo thời gian, AI trong game đã trải qua hai giai đoạn phát triển rõ rệt: từ các kỹ thuật lập trình hành vi cứng nhắc dựa trên quy tắc đến các phương pháp học máy có khả năng thích nghi và tự cải thiện.

2.1.1 *AI truyền thống*

Trong giai đoạn đầu, AI game chủ yếu dựa trên các hệ thống quy tắc cứng (rule-based systems), trong đó lập trình viên định nghĩa trước toàn bộ hành vi của NPC dưới dạng tập hợp các điều kiện và phản ứng tương ứng. Cách tiếp cận này dễ kiểm soát và dự đoán được, nhưng thiếu tính linh hoạt khi gặp các tình huống ngoài dự kiến.

Để khắc phục hạn chế đó, máy trạng thái hữu hạn (Finite State Machine - FSM) được ứng dụng rộng rãi trong những thập niên 1990–2000. FSM mô hình hóa hành vi của NPC thông qua một tập hữu hạn các trạng thái (tuần tra, truy đuổi, tấn công, rút lui) và các điều kiện chuyển trạng thái tương ứng. Nhờ tính đơn giản và hiệu quả tính toán cao, FSM vẫn được sử dụng phổ biến trong nhiều thể loại game đến tận ngày nay [9]. Tuy nhiên, khi số lượng trạng thái và điều kiện chuyển tiếp tăng lên, FSM nhanh chóng trở nên phức tạp và khó bảo trì, dẫn đến hiện tượng thường được gọi là “state explosion”.

Để giải quyết bài toán lập kế hoạch hành động phức tạp hơn, Goal-Oriented Action Planning (GOAP) được giới thiệu và phổ biến qua tựa game *F.E.A.R.* (2005) [9]. Thay vì định nghĩa cứng các chuyển trạng thái, GOAP cho phép NPC tự lập kế hoạch hành động dựa trên mục tiêu hiện tại và trạng thái thế giới, mang lại hành vi linh hoạt và tự nhiên hơn đáng kể. Dù vậy, toàn bộ tri thức trong GOAP vẫn phải được lập trình thủ công bởi con người, khiến chi phí xây dựng và mở rộng hệ thống tăng cao khi độ phức tạp của game tăng lên.

2.1.2 *AI học máy*

Sự bùng nổ của học máy trong thập niên 2010 đã mở ra một hướng tiếp cận hoàn toàn mới cho AI trong game. Thay vì lập trình hành vi thủ công, các kỹ thuật học máy cho phép agent tự học từ dữ liệu hoặc từ tương tác với môi trường. Trong

số các hướng tiếp cận đó, học tăng cường (Reinforcement Learning - RL) nổi lên như phương pháp phù hợp nhất với bản chất của game: agent học cách hành động thông qua thử và sai, nhận phần thưởng hoặc hình phạt từ môi trường, và dần dần tối ưu hóa chiến lược của mình để đạt điểm số cao nhất [19].

Bước ngoặt quan trọng đến vào năm 2013 khi DeepMind công bố thuật toán DQN (Deep Q-Network) [11], lần đầu tiên chứng minh một agent RL có thể học chơi thành thạo 49 trò chơi Atari khác nhau chỉ từ dữ liệu pixel thô mà không cần bất kỳ tri thức nào được lập trình sẵn. Thành tựu này đánh dấu sự ra đời của Deep Reinforcement Learning - kết hợp sức mạnh biểu diễn của mạng nơ-ron sâu với khả năng học từ tương tác của RL.

Những năm tiếp theo chứng kiến hàng loạt thành tựu ấn tượng: AlphaGo và AlphaGo Zero [17, 18] chinh phục cờ vây ở trình độ kiện tướng thế giới; OpenAI Five [12] đánh bại các đội chuyên nghiệp trong Dota 2; AlphaStar [20] đạt danh hiệu Grandmaster trong StarCraft II. Các kết quả này không chỉ có giá trị học thuật mà còn mở ra tiềm năng ứng dụng RL trực tiếp trong phát triển game thương mại, từ việc tạo NPC thông minh đến sinh nội dung tự động và hỗ trợ kiểm thử game [22].

2.2 Nền tảng Reinforcement Learning

Học tăng cường (Reinforcement Learning - RL) là một nhánh của học máy trong đó một tác nhân hay còn gọi là agent học cách hành động thông qua tương tác trực tiếp với môi trường. Khác với học có giám sát, RL không yêu cầu dữ liệu gán nhãn sẵn có; thay vào đó, agent tự khám phá không gian hành động và nhận phản hồi dưới dạng phần thưởng hoặc hình phạt để dần dần cải thiện chiến lược của mình [19]. Đây là đặc tính cốt lõi khiến RL trở nên phù hợp với các bài toán game, nơi hành vi tối ưu thường khó định nghĩa tường minh nhưng lại dễ đánh giá thông qua kết quả.

2.2.1 Các khái niệm cơ bản

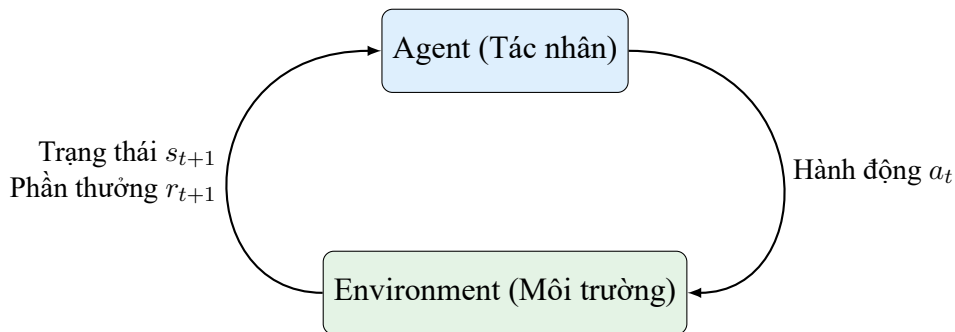
Vòng lặp tương tác trong RL được xây dựng xung quanh năm khái niệm nền tảng. Agent là thực thể ra quyết định - trong bối cảnh game, agent có thể là nhân vật người chơi, NPC hay robot mô phỏng.

Environment hay môi trường là thế giới mà agent tương tác, bao gồm toàn bộ các đối tượng, luật chơi và cơ chế phản hồi; trong đề tài này, môi trường là các scene 3D được xây dựng trong Unity.

State hay trạng thái $s \in \mathcal{S}$ là biểu diễn của môi trường tại một thời điểm nhất định mà agent có thể quan sát được. Trong môi trường 3D, state thường bao gồm vị trí, vận tốc của các đối tượng, khoảng cách đến mục tiêu và các thông tin không gian liên quan. Action hay hành động $a \in \mathcal{A}$ là tập hợp các lựa chọn mà agent có thể thực hiện tại mỗi bước, có thể là rời rạc như di chuyển lên, xuống, trái, phải hoặc liên tục như lực đẩy theo trục x, y, z .

Reward hay phần thưởng $r_t \in \mathbb{R}$ là tín hiệu phản hồi mà môi trường trả về sau mỗi hành động của agent, đóng vai trò là thước đo duy nhất để agent đánh giá chất lượng quyết định của mình. Việc thiết kế hàm phần thưởng hợp lý là một trong những bước quan trọng và khó nhất trong quá trình xây dựng hệ thống RL, vì một hàm phần thưởng không phù hợp có thể dẫn đến hành vi không mong muốn dù thuật toán hoạt động đúng về mặt lý thuyết [19].

Tại mỗi bước thời gian t , agent quan sát trạng thái s_t , thực hiện hành động a_t , nhận phần thưởng r_t và chuyển sang trạng thái mới s_{t+1} . Vòng lặp này lặp lại liên tục cho đến khi episode kết thúc, tức khi agent hoàn thành nhiệm vụ, thất bại hoặc vượt quá số bước tối đa, và hình thành nên một quỹ đạo tương tác $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$. Toàn bộ vòng lặp tương tác giữa agent và môi trường được minh họa trong Hình 2.1. Đây là sơ đồ khái quát nhất của mọi hệ thống học tăng cường: agent quan sát trạng thái hiện tại của môi trường, lựa chọn một hành động dựa trên chính sách của mình, và môi trường phản hồi lại bằng một trạng thái mới cùng một phần thưởng, cứ thế lặp đi lặp lại để agent dần cải thiện chính sách.



Hình 2.1: Vòng lặp tương tác giữa agent và môi trường trong học tăng cường

2.2.2 Markov Decision Process (MDP)

Hầu hết các bài toán RL được hình thức hóa dưới dạng Quá trình Quyết định Markov (Markov Decision Process — MDP), một khung toán học cung cấp nền tảng lý thuyết chặt chẽ cho việc mô hình hóa bài toán ra quyết định tuần tự trong môi trường ngẫu nhiên [13]. Một MDP được định nghĩa bởi bộ năm thành phần $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, trong đó $\mathcal{S}$ là không gian trạng thái, $\mathcal{A}$ là không gian hành động, $\mathcal{P}$ là hàm chuyển trạng thái, $\mathcal{R}$ là hàm phần thưởng và $\gamma \in [0, 1)$ là hệ số chiết khấu.

Hàm chuyển trạng thái $\mathcal{P}(s' \mid s, a)$ xác định xác suất chuyển từ trạng thái s sang trạng thái s' khi thực hiện hành động a . Tính chất Markov — hay còn gọi là tính không nhớ — đòi hỏi rằng trạng thái tiếp theo s_{t+1} chỉ phụ thuộc vào trạng thái hiện tại s_t và hành động a_t , không phụ thuộc vào lịch sử trước đó:

$$\mathcal{P}(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = \mathcal{P}(s_{t+1} \mid s_t, a_t) \quad (2.1)$$

Giả thiết Markov này rất quan trọng vì nó đảm bảo rằng agent chỉ cần quan sát trạng thái hiện tại để ra quyết định tối ưu, không cần lưu trữ toàn bộ lịch sử tương tác. Trong thực tế, nhiều môi trường game thỏa mãn gần đúng tính chất này nếu state được thiết kế đủ phong phú để mang đầy đủ thông tin cần thiết.

Hệ số chiết khấu γ xác định mức độ agent coi trọng phần thưởng tương lai so với phần thưởng hiện tại. Khi γ gần 1, agent có xu hướng lập kế hoạch dài hạn; khi γ nhỏ, agent tập trung vào phần thưởng tức thời. Tổng phần thưởng tích lũy có chiết khấu mà agent hướng đến tối đa hóa được định nghĩa là:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \quad (2.2)$$

2.2.3 Policy và Value Function

Policy (chính sách) π là hàm ánh xạ từ trạng thái sang hành động, định nghĩa chiến lược hành động của agent. Policy có thể là tất định $\pi : \mathcal{S} \rightarrow \mathcal{A}$ hoặc ngẫu nhiên $\pi(a \mid s) = P(A_t = a \mid S_t = s)$, cho biết xác suất chọn hành động a khi ở trạng thái s . Mục tiêu của RL là tìm policy tối ưu π^* sao cho tổng phần thưởng tích lũy kỳ vọng là lớn nhất.

Value Function hay còn được gọi là hàm giá trị đo lường mức độ tốt của một trạng thái hoặc một cặp trạng thái–hành động khi agent tuân theo một policy nhất định. Hàm giá trị trạng thái $V^\pi(s)$ được định nghĩa là tổng phần thưởng tích lũy kỳ vọng khi bắt đầu từ trạng thái s và tuân theo policy π :

$$V^\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid S_t = s \right] \quad (2.3)$$

Hàm giá trị hành động $Q^\pi(s, a)$ (hay Q-function) mở rộng khái niệm trên bằng cách đánh giá giá trị của việc thực hiện hành động a tại trạng thái s rồi sau đó tuân theo policy π :

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t \mid S_t = s, A_t = a] \quad (2.4)$$

Mối quan hệ giữa V^π và Q^π được thể hiện qua phương trình Bellman [1], nền tảng lý thuyết của hầu hết các thuật toán RL hiện đại:

$$V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s, a) \quad (2.5)$$

Policy tối ưu π^* và hàm giá trị tối ưu $V^*(s)$, $Q^*(s, a)$ thỏa mãn phương trình tối ưu Bellman:

$$Q^*(s, a) = \mathbb{E} \left[r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') \mid S_t = s, A_t = a \right] \quad (2.6)$$

Trong thực tế, với không gian trạng thái lớn và liên tục như các môi trường game 3D, việc biểu diễn chính xác V^π hay Q^π dưới dạng bảng tra cứu là không khả thi. Đây là động lực dẫn đến sự ra đời của Deep Reinforcement Learning, trong đó các hàm giá trị và policy được xấp xỉ bằng mạng nơ-ron sâu, được trình bày chi tiết trong Mục 2.3.

2.3 Học tăng cường sâu

Học tăng cường sâu (Deep Reinforcement Learning) là sự kết hợp giữa học tăng cường và mạng nơ-ron sâu (deep neural network), ra đời nhằm giải quyết hạn chế cốt lõi của RL truyền thống khi đối mặt với các bài toán có không gian trạng thái và hành động lớn, liên tục. Thay vì lưu trữ giá trị của từng cặp trạng thái–hành

động trong bảng tra cứu, Deep RL sử dụng mạng nơ-ron để xấp xỉ các hàm giá trị hoặc policy, cho phép tổng quát hóa sang các trạng thái chưa từng gặp trong quá trình huấn luyện [7]. Đây là đặc tính then chốt khiến Deep RL có thể xử lý được các môi trường game 3D phức tạp như trong đề tài này.

2.3.1 *Vai trò của mạng Neural Network trong RL*

Trong RL truyền thống, hàm giá trị $Q(s, a)$ hoặc policy $\pi(a | s)$ được biểu diễn dưới dạng bảng tra cứu (lookup table), đòi hỏi mỗi cặp (s, a) phải được thăm viếng đủ nhiều lần để ước lượng chính xác. Cách tiếp cận này chỉ khả thi với không gian trạng thái hữu hạn và nhỏ. Với các môi trường game 3D, không gian trạng thái có thể là liên tục và có số chiều rất lớn — ví dụ, vector quan sát của agent trong môi trường Football Table bao gồm vị trí, vận tốc của bóng và các thanh gạt trong không gian ba chiều — khiến bảng tra cứu hoàn toàn không khả thi.

Mạng nơ-ron sâu giải quyết vấn đề này bằng cách đóng vai trò là bộ xấp xỉ hàm tổng quát (universal function approximator). Thay vì lưu trữ giá trị cho từng trạng thái, mạng học một ánh xạ tham số hóa $f_\theta : \mathcal{S} \rightarrow \mathbb{R}$ (đối với value function) hoặc $f_\theta : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ (đối với policy), trong đó θ là tập tham số của mạng được tối ưu hóa qua quá trình huấn luyện. Nhờ khả năng tổng quát hóa, mạng có thể ước lượng giá trị hoặc đưa ra hành động cho các trạng thái chưa từng gặp dựa trên sự tương đồng với các trạng thái đã học [19].

Trong thực tế, kiến trúc mạng được lựa chọn tùy theo dạng dữ liệu quan sát. Với dữ liệu dạng vector số thực (vị trí, vận tốc, khoảng cách), mạng fully-connected nhiều lớp (MLP) là lựa chọn phổ biến và hiệu quả. Với dữ liệu dạng ảnh hoặc dữ liệu không gian, mạng tích chập (CNN) thường được ưu tiên. Unity ML-Agents hỗ trợ cả hai kiến trúc này và cho phép tùy chỉnh số lớp ẩn cũng như số đơn vị trong mỗi lớp thông qua tệp cấu hình YAML.

2.3.2 *Phân loại: Policy-Based, Value-Based, Actor-Critic*

Các thuật toán Deep RL hiện đại được phân loại thành ba nhóm chính dựa trên đối tượng mà mạng nơ-ron học để xấp xỉ, mỗi nhóm có ưu nhược điểm riêng và phù hợp với các dạng bài toán khác nhau.

Value-Based là nhóm thuật toán trong đó mạng nơ-ron học xấp xỉ hàm giá trị

$Q(s, a)$, từ đó suy ra policy tối ưu bằng cách chọn hành động có giá trị Q cao nhất. Đại diện tiêu biểu là DQN [11]. Nhóm này hoạt động tốt với không gian hành động rời rạc nhưng gặp khó khăn với không gian hành động liên tục do không thể tối ưu $\max_a Q(s, a)$ một cách hiệu quả khi $\mathcal{A}$ là liên tục.

Policy-Based là nhóm thuật toán trực tiếp tham số hóa và tối ưu policy $\pi_\theta(a | s)$ bằng cách tính gradient của hàm mục tiêu theo tham số θ thông qua phương pháp policy gradient [21]. Nhóm này xử lý tốt cả không gian hành động rời rạc lẫn liên tục, nhưng thường có phương sai cao trong ước lượng gradient, dẫn đến quá trình huấn luyện kém ổn định. PPO [16] thuộc nhóm này, bổ sung thêm cơ chế clipping để giới hạn độ lớn của mỗi bước cập nhật, giúp ổn định quá trình huấn luyện đáng kể.

Actor-Critic là nhóm kết hợp ưu điểm của cả hai nhóm trên bằng cách duy trì đồng thời hai mạng nơ-ron: mạng *actor* học policy $\pi_\theta(a | s)$ và mạng *critic* học hàm giá trị $V_\phi(s)$ hoặc $Q_\phi(s, a)$. Critic đóng vai trò đánh giá chất lượng hành động của actor, cung cấp tín hiệu học ổn định hơn so với việc chỉ dựa vào phần thưởng thực tế [10]. SAC [4] và MA-POCA [2] đều thuộc nhóm Actor-Critic, trong đó SAC bổ sung thêm entropy regularization để khuyến khích agent khám phá, còn MA-POCA mở rộng kiến trúc critic sang bài toán đa tác nhân với cơ chế centralized critic cho phép mỗi agent học dựa trên thông tin toàn cục trong quá trình huấn luyện.

Bảng 2.1 tóm tắt vị trí của ba thuật toán được sử dụng trong đề tài theo cách phân loại trên.

Bảng 2.1: Phân loại các thuật toán sử dụng trong đề tài

Thuật toán	Nhóm	On/Off-policy	Phù hợp với
PPO	Policy-Based	On-policy	Đối kháng, điều hướng
SAC	Actor-Critic	Off-policy	Hành động liên tục
MA-POCA	Actor-Critic	On-policy	Hợp tác đa tác nhân

2.4 Các thuật toán sử dụng trong đề tài

Đề tài sử dụng ba thuật toán học tăng cường đại diện cho các hướng tiếp cận khác nhau: PPO thuộc nhóm policy-based on-policy, SAC thuộc nhóm actor-critic off-policy và MA-POCA là thuật toán chuyên biệt cho bài toán đa tác nhân hợp tác.

Phần này trình bày nguyên lý hoạt động và đặc điểm của từng thuật toán, làm cơ sở cho việc lựa chọn và phân tích kết quả thực nghiệm ở các chương sau.

2.4.1 PPO (Proximal Policy Optimization)

PPO (Proximal Policy Optimization) được Schulman và cộng sự đề xuất vào năm 2017 [16] như một cải tiến của TRPO (Trust Region Policy Optimization) [15] nhằm giữ lại tính ổn định trong cập nhật policy nhưng đơn giản hơn đáng kể về mặt triển khai. PPO hiện là một trong những thuật toán RL phổ biến nhất trong cả nghiên cứu lẫn ứng dụng thực tế, và là thuật toán on-policy mặc định trong Unity ML-Agents.

Về nguyên lý hoạt động, PPO là thuật toán on-policy, nghĩa là dữ liệu huấn luyện được thu thập trực tiếp từ policy hiện tại và bị loại bỏ sau mỗi lần cập nhật. Ý tưởng cốt lõi của PPO là giới hạn độ lớn thay đổi của policy trong mỗi bước cập nhật, tránh tình trạng policy thay đổi quá đột ngột dẫn đến suy giảm hiệu năng không thể phục hồi - một vấn đề phổ biến trong các thuật toán policy gradient đơn giản.

PPO đạt được điều này thông qua hàm mục tiêu clipped surrogate:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right] \quad (2.7)$$

trong đó $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$ là tỷ lệ xác suất giữa policy mới và policy cũ, $\hat{A}_t$ là ước lượng advantage (mức độ hành động tốt hơn hoặc kém hơn kỳ vọng trung bình) và ε là siêu tham số clipping thường được đặt trong khoảng $[0.1, 0.2]$. Toán tử clip giới hạn $r_t(\theta)$ trong khoảng $[1 - \varepsilon, 1 + \varepsilon]$, ngăn policy mới lệch quá xa policy cũ trong một bước cập nhật. Hàm min đảm bảo mục tiêu luôn là cận dưới thận trọng (pessimistic lower bound) của mục tiêu không clipped.

Advantage $\hat{A}_t$ thường được ước lượng bằng Generalized Advantage Estimation (GAE):

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (2.8)$$

trong đó $\lambda \in [0, 1]$ là tham số cân bằng giữa bias và phương sai của ước lượng advantage, $V(s_t)$ là giá trị trạng thái được ước lượng bởi mạng critic.

Trong mỗi iteration, PPO thu thập một batch dữ liệu từ môi trường bằng policy hiện tại, sau đó thực hiện nhiều lần cập nhật gradient trên cùng batch đó (thường từ 3 đến 10 epochs) trước khi thu thập batch mới. Nhờ cơ chế clipping, các lần cập nhật lặp lại này không làm policy lệch quá xa điểm xuất phát, đảm bảo tính ổn định mà không cần tính toán tốn kém như TRPO.

Ưu điểm: PPO có tính ổn định huấn luyện cao nhờ cơ chế clipping, dễ triển khai và tinh chỉnh tham số, hoạt động tốt trên cả không gian hành động rời rạc lẫn liên tục. Thuật toán được tích hợp sẵn trong Unity ML-Agents với nhiều tham số được tối ưu theo kinh nghiệm, giúp rút ngắn đáng kể thời gian thiết lập thực nghiệm. PPO cũng dễ dàng mở rộng sang bài toán đa tác nhân thông qua cơ chế self-play có sẵn trong ML-Agents.

Nhược điểm: Do là thuật toán on-policy, PPO không tái sử dụng được dữ liệu cũ, dẫn đến hiệu quả mẫu (sample efficiency) thấp hơn so với các phương pháp off-policy. Điều này đặc biệt bất lợi trong các môi trường mà mỗi bước tương tác tốn nhiều thời gian tính toán. Ngoài ra, PPO nhạy cảm với việc lựa chọn siêu tham số như learning rate, số epoch và kích thước batch, đòi hỏi phải tinh chỉnh cẩn thận cho từng bài toán cụ thể.

2.4.2 SAC (Soft Actor-Critic)

SAC (Soft Actor-Critic) được Haarnoja và cộng sự đề xuất vào năm 2018 [4, 5] như một thuật toán actor-critic off-policy dựa trên nguyên lý maximum entropy, hướng đến giải quyết đồng thời hai thách thức lớn của RL: hiệu quả mẫu thấp và sự nhạy cảm với siêu tham số.

Điểm khác biệt cốt lõi của SAC so với các thuật toán RL thông thường nằm ở hàm mục tiêu. Thay vì chỉ tối đa hóa tổng phần thưởng kỳ vọng, SAC tối đa hóa đồng thời tổng phần thưởng và entropy của policy:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T \gamma^t \left(r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t)) \right) \right] \quad (2.9)$$

trong đó $\mathcal{H}(\pi(\cdot | s_t)) = -\mathbb{E}_{a \sim \pi} [\log \pi(a | s_t)]$ là entropy của policy tại trạng thái s_t và $\alpha > 0$ là hệ số nhiệt độ (temperature) cân bằng giữa phần thưởng và entropy. Entropy cao đồng nghĩa với policy ngẫu nhiên hơn, khuyến khích agent khám phá nhiều hành động khác nhau thay vì hội tụ sớm về một hành động tham lam.

SAC duy trì ba mạng nơ-ron: mạng actor $\pi_\theta(a | s)$ sinh ra phân phối xác suất trên không gian hành động, và hai mạng critic $Q_{\phi_1}(s, a)$, $Q_{\phi_2}(s, a)$ được huấn luyện độc lập để ước lượng hàm giá trị hành động. Việc sử dụng hai critic song song và lấy giá trị nhỏ hơn (clipped double-Q trick) giúp giảm thiểu hiện tượng overestimation phổ biến trong các thuật toán Q-learning:

$$Q_{\text{target}}(s_t, a_t) = r_t + \gamma \mathbb{E}_{a' \sim \pi} \left[\min_{i=1,2} Q_{\phi_i}(s_{t+1}, a') - \alpha \log \pi_\theta(a' | s_{t+1}) \right] \quad (2.10)$$

Là thuật toán off-policy, SAC lưu trữ toàn bộ kinh nghiệm tương tác vào replay buffer và lấy mẫu ngẫu nhiên (mini-batch) từ buffer này để cập nhật mạng, cho phép tái sử dụng dữ liệu cũ nhiều lần và đạt hiệu quả mẫu cao hơn đáng kể so với PPO. Hệ số nhiệt độ α có thể được cố định hoặc tự động điều chỉnh theo mục tiêu entropy mong muốn $\mathcal{H}_{\text{target}}$, giảm thiểu nhu cầu tinh chỉnh siêu tham số thủ công.

Ưu điểm: SAC có hiệu quả mẫu vượt trội so với các thuật toán on-policy nhờ replay buffer, đặc biệt phù hợp với không gian hành động liên tục - đặc điểm quan trọng trong môi trường Football Table của đề tài. Cơ chế entropy regularization giúp SAC tránh hội tụ sớm về cực tiểu cục bộ và duy trì khả năng khám phá trong suốt quá trình huấn luyện. Ngoài ra, SAC ít nhạy cảm hơn với việc lựa chọn learning rate so với PPO, mang lại tính ổn định tốt hơn trên nhiều bài toán khác nhau [5].

Nhược điểm: SAC đòi hỏi bộ nhớ lớn hơn để lưu trữ replay buffer và chi phí tính toán mỗi bước cập nhật cao hơn do phải huấn luyện đồng thời ba mạng nơ-ron. Do là thuật toán off-policy dựa trên replay buffer, SAC không tương thích trực tiếp với cơ chế self-play của môi trường đối kháng, vì khi đối thủ thay đổi thì các kinh nghiệm cũ trong buffer trở nên lỗi thời và gây nhiễu cho quá trình học. Đây cũng là một trong những giới hạn tích hợp sẵn của ML-Agents được phân tích chi tiết ở Mục 3.6.

2.4.3 MA-POCA (Multi-Agent POsthumous Credit Assignment)

MA-POCA được Cohen và cộng sự [2] đề xuất năm 2021 như thuật toán chuyên biệt cho bài toán học tăng cường đa tác nhân hợp tác (cooperative multi-agent), và là thuật toán đa tác nhân mặc định trong Unity ML-Agents. MA-POCA giải quyết hai thách thức đặc thù của môi trường đa tác nhân mà PPO hay SAC nguyên bản

khó xử lý tốt: bài toán *gán công lao* (credit assignment) — xác định đóng góp của từng agent vào phần thưởng chung của cả nhóm — và bài toán *tác nhân biến động* (posthumous credit) — gán công lao đúng cách cho những agent đã kết thúc hoạt động (bị loại hoặc hoàn thành nhiệm vụ) trước khi phần thưởng nhóm được trao.

Về nguyên lý hoạt động, MA-POCA mở rộng kiến trúc actor-critic theo mô hình *huấn luyện tập trung, thực thi phân tán* (centralized training with decentralized execution). Trong quá trình huấn luyện, một mạng critic tập trung (centralized critic) được phép quan sát trạng thái và hành động của *toàn bộ* các agent trong nhóm để ước lượng giá trị chung, nhờ đó đánh giá chính xác hơn đóng góp của từng agent. Tuy nhiên, khi triển khai thực tế, mỗi agent chỉ dựa trên quan sát cục bộ của riêng mình để ra quyết định, đảm bảo tính khả thi và khả năng mở rộng.

Điểm cải tiến cốt lõi của MA-POCA nằm ở việc sử dụng cơ chế attention để xử lý số lượng agent thay đổi động trong một episode. Thay vì cố định số lượng agent như các phương pháp trước [3, 8], MA-POCA cho phép agent gia nhập hoặc rời khỏi nhóm giữa chừng mà vẫn gán công lao chính xác — đặc tính rất quan trọng trong các game hợp tác nơi agent có thể “chết” hoặc hoàn thành nhiệm vụ ở các thời điểm khác nhau. Hàm giá trị nhóm được học dựa trên tổng phần thưởng nhóm (group reward) do môi trường cung cấp thông qua cơ chế SimpleMultiAgentGroup của ML-Agents.

Về phần cập nhật policy, MA-POCA kế thừa hàm mục tiêu clipped surrogate và cơ chế ước lượng advantage tương tự PPO (đều là thuật toán on-policy), nhưng thay hàm giá trị trạng thái đơn tác nhân $V(s)$ bằng hàm giá trị nhóm dựa trên baseline tập trung, cho phép mỗi agent nhận được tín hiệu học phản ánh đúng đóng góp của mình vào kết quả chung.

Ưu điểm: MA-POCA được thiết kế riêng cho bài toán hợp tác đa tác nhân, giải quyết hiệu quả vấn đề gán công lao mà PPO độc lập (mỗi agent học riêng lẻ) thường gặp phải. Cơ chế attention giúp thuật toán linh hoạt với số lượng agent thay đổi. Trong ML-Agents, MA-POCA tích hợp trực tiếp với cơ chế group reward, giúp việc thiết kế môi trường hợp tác trở nên tự nhiên và trực quan.

Nhược điểm: Là thuật toán on-policy, MA-POCA cũng chịu hạn chế về hiệu quả mẫu tương tự PPO. Kiến trúc centralized critic làm tăng độ phức tạp tính toán và bộ nhớ khi số lượng agent lớn. Ngoài ra, với bài toán đơn tác nhân, MA-POCA

gần như tương đương PPO nhưng phức tạp hơn không cần thiết, nên không mang lại lợi ích rõ rệt.

2.5 Unity ML-Agents Toolkit

Unity ML-Agents Toolkit [6] là một framework mã nguồn mở do Unity Technologies phát triển, cho phép xây dựng môi trường game và mô phỏng 3D làm nền tảng huấn luyện các agent thông minh bằng học tăng cường. Được giới thiệu lần đầu vào năm 2017 và liên tục cập nhật đến nay, ML-Agents đã trở thành một trong những nền tảng phổ biến nhất cho nghiên cứu RL trong môi trường game nhờ khả năng tích hợp liền mạch giữa engine đồ họa Unity và hệ sinh thái Python của cộng đồng học máy.

2.5.1 Kiến trúc tổng thể

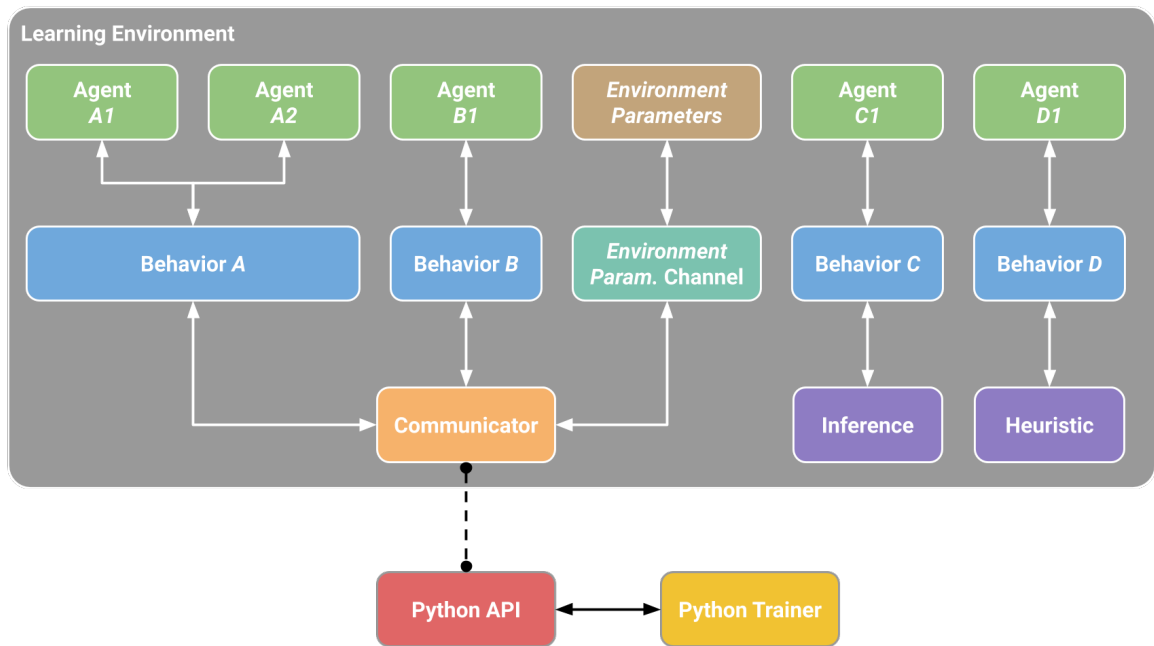
Kiến trúc của ML-Agents được xây dựng theo mô hình client-server, trong đó Unity Engine đóng vai trò môi trường mô phỏng và Python đóng vai trò backend huấn luyện. Hai thành phần này giao tiếp với nhau thông qua giao thức gRPC qua socket, cho phép chạy đồng thời nhiều instance môi trường Unity để tăng tốc thu thập dữ liệu.

Về phía Unity, thành phần trung tâm là Agent - một MonoBehaviour được gắn vào GameObject trong scene, chịu trách nhiệm thu thập quan sát CollectObservations, nhận hành động OnActionReceived và tính toán phần thưởng. Mỗi Agent được liên kết với một BehaviorParameters component định nghĩa không gian quan sát, không gian hành động và loại hành vi (học hoặc heuristic). Academy là singleton quản lý toàn bộ vòng lặp huấn luyện, đồng bộ hóa các bước thời gian giữa tất cả các agent trong scene và điều phối giao tiếp với Python backend.

Về phía Python, mlagents-learn là điểm vào chính của quá trình huấn luyện, tiếp nhận tệp cấu hình YAML và khởi tạo trainer tương ứng. Các lớp Trainer (PPOTrainer, POCATrainer) thực hiện logic huấn luyện, bao gồm thu thập trajectory, tính toán advantage, cập nhật tham số mạng và lưu checkpoint. Thư viện mlagents_envs cung cấp Python API cấp thấp cho phép tương tác trực tiếp với môi trường Unity mà không cần đi qua trainer tích hợp sẵn.

Hình 2.2 minh họa kiến trúc tổng thể của hệ thống ML-Agents và luồng dữ liệu

giữa các thành phần.



Hình 2.2: Kiến trúc tổng thể của Unity ML-Agents Toolkit

2.5.2 Quy trình huấn luyện Agent

Quy trình huấn luyện một agent trong ML-Agents diễn ra theo vòng lặp lặp lại liên tục cho đến khi đạt số bước tối đa hoặc người dùng dừng thủ công. Tại mỗi bước thời gian, Unity thu thập quan sát từ tất cả các agent đang hoạt động trong scene, gửi dữ liệu sang Python backend thông qua giao thức gRPC. Python trainer xử lý quan sát, suy luận hành động từ policy hiện tại (hoặc lấy mẫu ngẫu nhiên trong giai đoạn khởi động) và trả hành động về Unity. Unity thực thi hành động, cập nhật trạng thái vật lý, tính toán phần thưởng và gửi kết quả trở lại Python để tích lũy vào buffer.

Sau khi thu thập đủ một batch dữ liệu (kích thước được cấu hình qua tham số `buffer_size`), trainer thực hiện cập nhật tham số mạng. Đối với PPO, quá trình cập nhật gồm nhiều epoch trên cùng batch với hàm mục tiêu clipped surrogate như đã trình bày tại Mục 2.4.1. Sau khi cập nhật xong, buffer được xóa và quá trình thu thập dữ liệu mới bắt đầu. Các chỉ số huấn luyện như cumulative reward, episode length và policy loss được ghi lại liên tục và có thể theo dõi theo thời gian thực qua TensorBoard.

Tệp cấu hình YAML đóng vai trò trung tâm trong việc kiểm soát toàn bộ quá

trình huấn luyện. Tập này định nghĩa loại trainer (`trainer_type`), các siêu tham số huấn luyện (`hyperparameters`), kiến trúc mạng nơ-ron (`network_settings`), cấu hình tín hiệu phần thưởng (`reward_signals`) và số bước huấn luyện tối đa (`max_steps`). Mỗi behavior trong scene tương ứng với một khối cấu hình riêng, cho phép huấn luyện đồng thời nhiều loại agent với các thiết lập khác nhau trong cùng một session. Cấu hình chi tiết cho từng thuật toán được trình bày trong Phụ lục A, B và C.

2.5.3 Hỗ trợ thuật toán và giới hạn tích hợp sẵn

Unity ML-Agents tích hợp sẵn hai trainer chính thức: PPO cho bài toán đơn tác nhân và đối kháng (kết hợp self-play), và POCA (MA-POCA) cho bài toán hợp tác đa tác nhân. Cả hai đều là thuật toán on-policy. Bên cạnh đó, ML-Agents cũng cung cấp trainer SAC như một lựa chọn off-policy, cùng các tín hiệu phần thưởng nội tại như *curiosity* và *RND* để hỗ trợ khám phá trong môi trường phần thưởng thưa. Người dùng lựa chọn thuật toán và cấu hình siêu tham số hoàn toàn thông qua tệp YAML mà không cần thay đổi mã nguồn C# của môi trường.

Tuy nhiên, việc tích hợp sẵn này cũng đi kèm một số giới hạn quan trọng đối với đề tài. Thứ nhất, cơ chế self-play — vốn cần thiết cho môi trường đối kháng như Football Table — chỉ được hỗ trợ chính thức cho PPO và POCA, mà không hỗ trợ cho SAC. Nguyên nhân là replay buffer của SAC lưu trữ kinh nghiệm từ các phiên bản đối thủ cũ, trở nên lỗi thời và mâu thuẫn khi đối thủ liên tục được cập nhật trong quá trình self-play. Thứ hai, cơ chế group reward phục vụ bài toán hợp tác được thiết kế tối ưu cho POCA; khi áp dụng cho PPO hoặc SAC, phần thưởng nhóm buộc phải được phân bổ thủ công về từng agent, làm suy giảm khả năng gán công lao. Thứ ba, các trainer tích hợp sẵn không cho phép người dùng dễ dàng can thiệp sâu vào vòng lặp huấn luyện (ví dụ tùy biến hàm mất mát hay lịch trình cập nhật).

Để so sánh công bằng cả ba thuật toán PPO, SAC và MA-POCA trên môi trường đối kháng, đề tài cần một hướng tiếp cận vượt ra ngoài trainer tích hợp sẵn của ML-Agents. Thư viện `mlagents_envs` cung cấp Python Low-Level API cho phép điều khiển trực tiếp môi trường Unity như một môi trường kiểu Gym, từ đó kết nối với các thư viện RL bên ngoài như Stable-Baselines3 [14]. Cách tiếp cận

này được trình bày chi tiết trong Mục 3.6.

3. THIẾT KẾ MÔI TRƯỜNG VÀ PHƯƠNG PHÁP THỰC NGHIỆM

Chương này trình bày chi tiết quá trình hiện thực hóa ý tưởng của đề tài thành một hệ thống thực nghiệm hoàn chỉnh. Nội dung được tổ chức theo trình tự từ tổng quan đến cụ thể: trước hết là kiến trúc chung của toàn bộ hệ thống huấn luyện cùng những nguyên tắc thiết kế được áp dụng thống nhất cho cả ba môi trường; tiếp theo là phần mô tả chi tiết từng môi trường, gồm bài toán, thiết kế tác nhân, hàm phần thưởng và lựa chọn thuật toán; sau đó là bảng tổng hợp đối chiếu ba môi trường. Cuối chương, đề tài trình bày phương pháp tích hợp thuật toán ra ngoài bộ huấn luyện tích hợp sẵn của ML-Agents, cùng toàn bộ thiết lập thực nghiệm và tiêu chí đánh giá.

3.1 Tổng quan thiết kế hệ thống

3.1.1 Kiến trúc huấn luyện tổng thể

Toàn bộ hệ thống thực nghiệm được xây dựng theo kiến trúc client-server của Unity ML-Agents đã trình bày ở Mục 2.5.1. Trong kiến trúc này, Unity đảm nhận vai trò mô phỏng: nó dựng nên thế giới ba chiều, tính toán vật lý va chạm, thu thập quan sát của tác nhân và thực thi các hành động mà tác nhân lựa chọn. Phía Python, thông qua chương trình `mlagents-learn`, đảm nhận vai trò huấn luyện: nó nhận quan sát từ Unity, suy luận ra hành động dựa trên chính sách hiện tại và cập nhật tham số của mạng nơ-ron dựa trên phần thưởng thu về. Hai phía trao đổi dữ liệu với nhau theo từng bước thời gian thông qua giao thức truyền thông nội bộ.

Ba môi trường của đề tài được tổ chức thành ba khu vực độc lập trong cùng một dự án Unity, tương ứng với ba thư mục `Assets/Football`, `Assets/Collaboration` và `Assets/CrossTheRoad`. Mỗi môi trường được gán với một loại hành vi riêng, lần lượt mang tên `Football`, `PuzzleBehavior` và `CrossTheRoad`. Việc tách biệt này cho phép huấn luyện từng môi trường một cách độc lập mà không gây nhiễu lẫn nhau, đồng thời giúp việc nạp đúng tệp cấu hình cho từng lần chạy trở nên rõ ràng.

Một đặc điểm quan trọng trong thiết kế là mỗi cảnh chứa nhiều bản sao của môi

trường được đặt cạnh nhau và chạy song song. Nhờ đó, tại mỗi bước thời gian, hệ thống thu thập được nhiều mẫu kinh nghiệm cùng lúc, giúp tăng tốc đáng kể quá trình huấn luyện. Bên cạnh đó, tham số tăng tốc mô phỏng được đặt ở mức gấp hai mươi lần thời gian thực, cho phép Unity mô phỏng nhanh hơn nhiều so với tốc độ hiển thị thông thường mà không ảnh hưởng đến tính đúng đắn của vật lý. Toàn bộ chín lần huấn luyện, tương ứng với ba môi trường nhân ba thuật toán, được điều phối tự động bằng một chương trình Python được trình bày ở Mục 3.6.3.

3.1.2 *Không gian quan sát*

Không gian quan sát quyết định những gì tác nhân có thể “nhìn thấy” về môi trường tại mỗi thời điểm, và do đó ảnh hưởng trực tiếp đến chất lượng chính sách mà tác nhân có thể học được. Cả ba môi trường của đề tài đều sử dụng dạng quan sát vector, tức một chuỗi số thực mô tả các đại lượng liên quan như vị trí, vận tốc, khoảng cách hay trạng thái của các đối tượng. Nguyên tắc thiết kế xuyên suốt là chỉ đưa vào những thông tin tối thiểu nhưng đủ để tác nhân thỏa mãn gần đúng tính chất Markov, nghĩa là chỉ cần quan sát hiện tại đã đủ để ra quyết định hợp lý mà không cần ghi nhớ toàn bộ lịch sử.

Để quá trình huấn luyện ổn định, các giá trị quan sát đều được chuẩn hóa về lân cận khoảng từ âm một đến một trước khi đưa vào mạng nơ-ron. Việc chuẩn hóa này đặc biệt cần thiết với môi trường Football Table, nơi các đại lượng vật lý như vận tốc bóng và góc xoay thanh gạt có thang đo rất khác nhau; nếu không chuẩn hóa, những đại lượng có giá trị lớn sẽ lấn át các đại lượng nhỏ và làm sai lệch quá trình học. Ngoài quan sát vector, hai môi trường Cross The Road và Cooperative Puzzle còn được trang bị thêm cảm biến tia. Cảm biến này phóng ra một chùm tia từ vị trí tác nhân theo nhiều hướng và ghi lại khoảng cách cùng loại vật thể mà mỗi tia chạm tới, qua đó mô phỏng khả năng quan sát không gian xung quanh tương tự như thị giác của một thực thể trong thế giới ba chiều.

3.1.3 *Không gian hành động*

Đề tài chủ ý lựa chọn ba môi trường có không gian hành động khác nhau nhằm đánh giá tính linh hoạt của từng thuật toán trên nhiều dạng điều khiển. Môi trường Football Table sử dụng không gian hành động liên tục gồm tám chiều, tương ứng với lực trượt ngang và lực xoay của bốn thanh gạt; mỗi chiều nhận một giá trị thực

trong một khoảng liên tục, phản ánh bản chất điều khiển vật lý mượt mà của trò chơi bi lắc. Hai môi trường còn lại sử dụng không gian hành động rời rạc, trong đó tác nhân chọn một trong một số hữu hạn các hành động định sẵn tại mỗi bước. Môi trường Cross The Road có bốn hành động là đứng yên, sang trái, sang phải và tiến về phía trước, còn môi trường Cooperative Puzzle có bảy hành động gồm đứng yên, tiến, lùi, xoay trái, xoay phải, đi ngang sang trái và đi ngang sang phải. Sự có mặt đồng thời của cả hành động liên tục lẫn rời rạc cho phép làm nổi bật thế mạnh của SAC trên không gian liên tục, đồng thời kiểm chứng khả năng của cả ba thuật toán trên không gian rời rạc.

3.1.4 Nguyên tắc thiết kế hàm phần thưởng

Hàm phần thưởng là thành phần khó thiết kế nhất trong một hệ thống học tăng cường, bởi nó là tín hiệu duy nhất định hướng hành vi của tác nhân. Một hàm phần thưởng thiết kế không tốt có thể khiến tác nhân học được những hành vi kỳ lạ nhưng vẫn tối đa hóa phần thưởng, một hiện tượng thường được gọi là lợi dụng phần thưởng. Để tránh điều này, đề tài áp dụng nhất quán nguyên tắc kết hợp giữa phần thưởng thưa tại trạng thái kết thúc và phần thưởng định hướng dày đặc.

Phần thưởng thưa, chẳng hạn cộng một điểm khi ghi bàn hoặc khi đến đích, xác định rõ mục tiêu cuối cùng mà tác nhân cần đạt được. Tuy nhiên, nếu chỉ dựa vào phần thưởng thưa thì trong giai đoạn đầu, khi tác nhân hầu như chưa bao giờ tình cờ đạt mục tiêu, tín hiệu học sẽ quá hiếm và quá trình huấn luyện tiến triển rất chậm. Vì vậy, đề tài bổ sung thêm các phần thưởng định hướng nhỏ, chẳng hạn thưởng khi tác nhân đá bóng về phía khung thành đối phương, để cung cấp tín hiệu học thường xuyên hơn và dẫn dắt tác nhân đi đúng hướng. Điều cốt yếu là độ lớn của các phần thưởng định hướng luôn được giữ nhỏ hơn nhiều so với phần thưởng mục tiêu, để chúng chỉ đóng vai trò gợi ý chứ không làm tác nhân sao nhãng khỏi mục tiêu chính. Chi tiết hàm phần thưởng của từng môi trường được trình bày ngay trong các mục tương ứng dưới đây.

3.2 Môi trường 1: Football Table

3.2.1 *Mô tả bài toán và môi trường*

Môi trường Football Table mô phỏng một bàn bi lắc ba chiều với đầy đủ vật lý và chạm giữa quả bóng và các cầu thủ gắn trên thanh gạt. Đây là bài toán đối kháng một đối một, trong đó hai đội chơi đối đầu trực tiếp; mỗi đội điều khiển bốn thanh gạt với mục tiêu đưa bóng vào khung thành đối phương đồng thời bảo vệ khung thành của mình. Điểm đặc trưng của bài toán nằm ở tính đối kháng và ở động lực học liên tục của quả bóng: bóng có thể lăn, nảy và đổi hướng liên tục dưới tác động của va chạm, buộc tác nhân phải phản ứng nhanh và phải học được cả kỹ năng tấn công lẫn phòng thủ. Chính tính đối kháng khiến bài toán này trở thành trường hợp điển hình để kiểm chứng cơ chế tự chơi, trong đó tác nhân tiến bộ bằng cách liên tục thi đấu với những phiên bản ngày càng mạnh hơn của chính nó.

3.2.2 *Thiết kế tác nhân và luật chơi*

Mỗi đội được điều khiển bởi một tác nhân duy nhất, quản lý đồng thời cả bốn thanh gạt của đội. Tại mỗi bước, tác nhân xuất ra tám giá trị liên tục, trong đó với mỗi thanh gạt có một giá trị điều khiển chuyển động trượt ngang để đưa cầu thủ đến đúng vị trí, và một giá trị điều khiển chuyển động xoay để thực hiện cú đá. Cách gộp toàn bộ đội vào một tác nhân giúp đơn giản hóa việc phối hợp giữa các thanh gạt, đồng thời phù hợp với cơ chế tự chơi khi mỗi phía chỉ cần một chính sách.

Vector quan sát của tác nhân bao gồm ba nhóm thông tin. Nhóm thứ nhất mô tả trạng thái quả bóng, gồm vận tốc và vị trí của bóng trong không gian ba chiều sau khi đã chuẩn hóa. Nhóm thứ hai mô tả trạng thái đội nhà, trong đó với mỗi thanh gạt, tác nhân quan sát được vận tốc trượt, vận tốc xoay, vị trí và góc xoay hiện tại. Nhóm thứ ba mô tả trạng thái đội đối phương với các đại lượng tương tự. Toàn bộ các giá trị này được nhân với một hệ số dấu tùy theo phía của đội, nhờ đó một chính sách duy nhất có thể vận hành đối xứng cho cả hai phía bàn mà không cần huấn luyện riêng.

Một lượt chơi kết thúc khi có bàn thắng, tức khi bóng đi vào một trong hai khung thành. Để tạo ra đối thủ trong quá trình huấn luyện, môi trường sử dụng cơ chế tự chơi: tác nhân thi đấu với chính các phiên bản trước đó của mình được lưu lại theo định kỳ, nhờ đó trình độ của cả hai phía cùng tăng lên theo thời gian và

tạo thành một chương trình học tự nhiên với độ khó tăng dần. Bên cạnh đó, môi trường còn được trang bị một đối thủ điều khiển bằng luật đơn giản, hoạt động dựa trên vị trí quả bóng, để phục vụ việc huấn luyện những thuật toán không hỗ trợ cơ chế tự chơi cũng như để làm mốc đánh giá chung cho cả ba thuật toán.

3.2.3 Hàm phần thưởng

Hàm phần thưởng của môi trường Football Table được tổng hợp trong Bảng 3.1. Thành phần chính là phần thưởng thừa: tác nhân được cộng một điểm khi ghi bàn và bị trừ một điểm khi bị thủng lưới, qua đó xác lập rõ mục tiêu thắng thua của trận đấu. Hai thành phần định hướng được bổ sung để tăng tốc quá trình học ở giai đoạn đầu. Thành phần thứ nhất thưởng cho tác nhân khi nó đá bóng theo hướng về phía khung thành đối phương, với độ lớn tỉ lệ thuận với mức độ trùng khớp giữa hướng bóng và hướng tới khung thành; thành phần này chỉ được kích hoạt khi tác nhân đang kiểm soát bóng. Thành phần thứ hai là một hình phạt nhỏ khi tác nhân xoay thanh gạt quá mạnh, nhằm hạn chế hành vi xoay tít vô nghĩa vốn có thể tình cờ ghi bàn nhưng không phản ánh kỹ năng thực sự.

Bảng 3.1: Hàm phần thưởng của môi trường Football Table

Sự kiện	Giá trị	Loại
Ghi bàn vào khung thành đối phương	+1,0	Thưởng mục tiêu
Bị thủng lưới	-1,0	Phạt mục tiêu
Đá bóng hướng về khung thành đối phương	$+0,1 \times \cos \theta$	Định hướng
Xoay thanh gạt quá mức	$-0,01 \times \text{độ xoay}$	Phạt định hướng

trong đó θ là góc giữa hướng chuyển động của bóng và hướng nối từ bóng tới khung thành đối phương.

3.2.4 Lựa chọn và cấu hình thuật toán

Do bản chất đối kháng và không gian hành động liên tục, môi trường này được huấn luyện chính bằng PPO kết hợp cơ chế tự chơi, đây là lựa chọn tự nhiên nhất và được ML-Agents hỗ trợ đầy đủ. Mạng nơ-ron sử dụng ba lớp ẩn với năm trăm mười hai đơn vị mỗi lớp, đủ dung lượng để nắm bắt động lực học phức tạp của quả

bóng, và bất chế độ chuẩn hóa quan sát do các đại lượng vật lý có thang đo khác nhau. Cơ chế tự chơi được cấu hình để lưu lại các phiên bản đối thủ theo định kỳ và theo dõi tiến bộ qua chỉ số xếp hạng, khởi tạo ở mức một nghìn hai trăm. Hai thuật toán còn lại được đưa vào để so sánh: SAC huấn luyện bằng cách đấu với đối thủ luật cố định do không hỗ trợ tự chơi, còn MA-POCA vận hành với phê bình tập trung nhưng không sử dụng phần thưởng nhóm. Cấu hình siêu tham số đầy đủ của cả ba thuật toán được liệt kê trong các Phụ lục A, B và C.

3.3 Môi trường 2: Cooperative Puzzle (hợp tác đa tác nhân)

3.3.1 Mô tả bài toán và môi trường

Môi trường thứ hai là một bài toán giải đồ mang tính hợp tác, trong đó hai tác nhân phải phối hợp chặt chẽ để cùng vượt qua một câu đố không gian. Điểm mấu chốt của bài toán là nhiệm vụ được thiết kế sao cho không một tác nhân nào có thể tự mình hoàn thành: cả hai phải cùng đứng lên các bàn đạp áp lực để mở cửa, cùng rời khỏi căn phòng ban đầu và cuối cùng cùng đến được điểm đích. Sự phụ thuộc lẫn nhau này biến bài toán thành một trường hợp điển hình để làm nổi bật thách thức gán công lao trong học tăng cường hợp tác, tức việc xác định đóng góp thực sự của từng tác nhân vào thành quả chung của cả nhóm. Đây cũng chính là dạng bài toán mà MA-POCA được thiết kế để giải quyết.

3.3.2 Thiết kế tác nhân và cơ chế phối hợp

Mỗi tác nhân điều khiển một robot di chuyển trong không gian ba chiều với bảy hành động rời rạc gồm đứng yên, tiến, lùi, xoay trái, xoay phải, đi ngang sang trái và đi ngang sang phải. Tập hành động này đủ phong phú để robot vừa di chuyển linh hoạt vừa định hướng chính xác khi cần đứng vào đúng vị trí bàn đạp. Về quan sát, tác nhân nhận biết trạng thái đã được nhấn hay chưa của hai bàn đạp áp lực, trạng thái đã tìm thấy điểm đích hay chưa, kết hợp với cảm biến tia để xác định vị trí của đồng đội, các bức tường và các mục tiêu xung quanh.

Cơ chế phối hợp được hiện thực thông qua một bộ điều khiển môi trường sử dụng nhóm đa tác nhân của ML-Agents. Cả hai tác nhân được đăng ký vào cùng một nhóm, và phần thưởng nhóm được trao cho toàn nhóm khi các điều kiện hợp tác được thỏa mãn. Chính cách tổ chức này cho phép huấn luyện hành vi phối hợp

thực sự, thay vì để mỗi tác nhân chỉ tối ưu lợi ích riêng của mình. Bên cạnh phần thưởng nhóm, hệ thống còn có các khoản phạt nhóm khi một tác nhân hành động lệch nhịp với đồng đội, chẳng hạn khi một tác nhân đứng lên bàn đạp trong lúc đồng đội vẫn còn kẹt lại trong phòng; những hình phạt này buộc hai tác nhân phải học cách hành động đồng bộ với nhau.

3.3.3 Hàm phần thưởng

Hệ thống phần thưởng của môi trường, được tổng hợp trong Bảng 3.2, kết hợp giữa phần thưởng nhóm và phần thưởng cá nhân định hướng. Đáng chú ý, phần lớn các khoản thưởng và phạt định hướng đều được chia cho số bước tối đa của một lượt chơi, nhờ đó tổng phần thưởng tích lũy qua toàn bộ lượt chơi được giữ ở một thang đo hợp lý và không bị phình to quá mức. Phần thưởng lớn nhất, bằng một điểm cho cả nhóm, chỉ được trao khi cả hai tác nhân cùng đến được điểm đích, qua đó nhấn mạnh rằng thành công chỉ có ý nghĩa khi cả nhóm cùng hoàn thành. Các khoản thưởng nhỏ khuyến khích những hành vi trung gian đúng hướng như đứng lên bàn đạp, rời khỏi phòng hay đẩy khối vật cản, trong khi các khoản phạt xử lý những hành vi lệch nhịp và một khoản phạt thời gian nhỏ ở mỗi bước thúc đẩy nhóm hoàn thành nhiệm vụ nhanh hơn.

Bảng 3.2: Hàm phần thưởng của môi trường Cooperative Puzzle

Sự kiện	Giá trị	Phạm vi
Cả hai tác nhân đến điểm đích	+1,0	Nhóm
Tác nhân đứng trên bàn đạp áp lực	+0,25/số bước	Cá nhân
Tác nhân đã rời khỏi phòng ban đầu	+0,5/số bước	Cá nhân
Cả hai tác nhân cùng rời phòng	+0,5/số bước	Nhóm
Tác nhân đẩy khối vật cản	+0,25/số bước	Cá nhân
Hành động lệch nhịp với đồng đội	từ -2,0 đến -4,0 chia số bước	Nhóm
Phạt thời gian mỗi bước	-0,25/số bước	Nhóm

3.3.4 Lựa chọn và cấu hình thuật toán

Vì môi trường sử dụng cơ chế phần thưởng nhóm, MA-POCA là thuật toán phù hợp nhất về mặt lý thuyết và được kỳ vọng đạt kết quả tốt nhất. Mạng nơ-ron sử

dụng ba lớp ẩn với hai trăm năm mươi sáu đơn vị mỗi lớp, kèm theo một tín hiệu tò mò với cường độ nhỏ để khuyến khích tác nhân khám phá trong môi trường có phần thưởng thưa. Để so sánh, đề tài cũng huấn luyện PPO, trong đó mỗi tác nhân học độc lập và phần thưởng nhóm được phân bổ thủ công về từng tác nhân, cùng với SAC, trong đó mỗi tác nhân sử dụng một bộ đệm kinh nghiệm riêng. Số bước huấn luyện tối đa cho môi trường này được đặt ở mức năm triệu bước nhằm bảo đảm đủ thời gian cho hành vi phối hợp phức tạp hình thành.

3.4 Môi trường 3: Vượt đường tránh phương tiện

3.4.1 Mô tả bài toán và môi trường

Môi trường Cross The Road là bài toán điều hướng đơn tác nhân, trong đó một tác nhân phải băng qua một con đường có các phương tiện di chuyển liên tục để đến được vạch đích ở phía bên kia mà không bị va chạm. Dù đơn giản về hình thức, đây lại là mô hình thu nhỏ của rất nhiều tình huống điều hướng tránh vật cản động thường gặp trong game, chẳng hạn nhân vật phải né tránh đạn, né tránh kẻ địch hay vượt qua các chướng ngại chuyển động. Thách thức cốt lõi nằm ở chỗ tác nhân phải học được thời điểm nào nên di chuyển và thời điểm nào nên chờ đợi, bởi lao lên đúng lúc có phương tiện đi qua sẽ dẫn đến va chạm, còn chờ đợi quá lâu lại không đạt được mục tiêu.

3.4.2 Thiết kế tác nhân và chướng ngại vật

Tác nhân di chuyển theo từng ô lưới với bốn hành động rời rạc là đứng yên, sang trái, sang phải và tiến về phía trước; mỗi hành động di chuyển đưa tác nhân đi một khoảng cố định. Vector quan sát gồm sáu giá trị, tương ứng với tọa độ ba chiều của tác nhân và tọa độ ba chiều của điểm đích, kết hợp với cảm biến tia để phát hiện các phương tiện đang tiến lại gần. Sự kết hợp giữa thông tin vị trí tuyệt đối và thông tin cảm biến khoảng cách cho phép tác nhân vừa biết mình cần đi về hướng nào, vừa nhận biết nguy cơ va chạm tức thời.

Các chướng ngại vật là những phương tiện di chuyển qua lại trên các làn đường với tốc độ và thời điểm xuất hiện thay đổi, tạo nên tính động và tính ngẫu nhiên cho môi trường. Chính sự thay đổi này ngăn tác nhân học vẹt một chuỗi hành động cố định, buộc nó phải học một chính sách phản ứng thực sự dựa trên tình huống

quan sát được. Khi tác nhân va chạm với một phương tiện, lượt chơi kết thúc kèm một hình phạt; ngược lại, khi tác nhân chạm được vạch đích, lượt chơi kết thúc kèm một phần thưởng.

3.4.3 Hàm phần thưởng

Hàm phần thưởng của môi trường, trình bày trong Bảng 3.3, được thiết kế theo hướng đơn giản và trực tiếp: một phần thưởng lớn khi đến đích và một hình phạt nhỏ khi va chạm. Điểm đáng lưu ý là hình phạt va chạm được giữ ở mức rất nhỏ. Nếu hình phạt va chạm quá lớn, tác nhân có xu hướng trở nên quá thận trọng và chọn cách đứng yên vĩnh viễn để tránh mọi rủi ro, một hành vi tuy an toàn nhưng không bao giờ hoàn thành nhiệm vụ. Việc giữ hình phạt nhỏ, kết hợp với một tín hiệu tò mò khuyến khích khám phá, giúp tác nhân duy trì được động lực tiến về phía trước trong khi vẫn học được cách né tránh.

Bảng 3.3: Hàm phần thưởng của môi trường Cross The Road

Sự kiện	Giá trị	Loại
Đến được vạch đích	+1,0	Thưởng mục tiêu
Va chạm với phương tiện	-0,025	Phạt mục tiêu
Tín hiệu tò mò khuyến khích khám phá	cường độ 0,02	Nội tại

3.4.4 Lựa chọn và cấu hình thuật toán

Đây là môi trường đơn tác nhân với không gian hành động rời rạc nhỏ, rất phù hợp để so sánh trực tiếp cả ba thuật toán trên cùng một bài toán trong điều kiện tương đương. Mạng nơ-ron sử dụng hai lớp ẩn với một trăm hai mươi tám đơn vị mỗi lớp, một kích thước vừa phải tương xứng với độ phức tạp của bài toán. PPO và MA-POCA sử dụng cấu hình on-policy tương đương nhau về kích thước lô và bộ đệm, trong khi SAC sử dụng một bộ đệm kinh nghiệm lớn để khai thác tối đa lợi thế tái sử dụng dữ liệu của cơ chế off-policy. Số bước huấn luyện tối đa được đặt ở mức hai triệu bước cho cả ba thuật toán, bảo đảm một mặt bằng so sánh công bằng.

3.5 Bảng tổng hợp đặc trưng ba môi trường

Ba môi trường vừa trình bày tuy khác nhau về hình thức nhưng được lựa chọn một cách có chủ đích để bao phủ ba dạng bài toán học tăng cường điển hình, đồng thời tương ứng với ba dạng cơ chế thường gặp trong game. Bảng 3.4 tổng hợp và đối chiếu nhanh các đặc trưng chính của ba môi trường, làm cơ sở cho việc phân tích kết quả ở Chương 4. Có thể thấy mỗi môi trường không chỉ khác nhau ở dạng bài toán và số lượng tác nhân, mà còn khác nhau ở dạng không gian hành động, ở việc có hay không sử dụng phần thưởng nhóm, và ở cơ chế đặc thù đi kèm.

Bảng 3.4: Tổng hợp đặc trưng ba môi trường thực nghiệm

Đặc trưng	Football Table	Cooperative Puzzle	Cross The Road
Dạng bài toán	Đối kháng một đối một	Hợp tác đa tác nhân	Điều hướng đơn tác nhân
Số tác nhân	Hai, đối kháng	Hai, hợp tác	Một
Không gian hành động	Liên tục, tám chiều	Rời rạc, bảy hành động	Rời rạc, bốn hành động
Quan sát	Bóng và hai đội	Bàn đạp và cảm biến tia	Vị trí và cảm biến tia
Phần thưởng nhóm	Không	Có	Không
Cơ chế đặc thù	Tự chơi	Phần thưởng nhóm	Vật cản động
Số bước tối đa	Mười triệu	Năm triệu	Hai triệu
Thuật toán kỳ vọng tốt nhất	PPO tự chơi	MA-POCA	SAC

3.6 Triển khai thuật toán ngoài ML-Agents

3.6.1 Vấn đề với SAC trong ML-Agents

Như đã phân tích ở Mục 2.5.3, bộ huấn luyện SAC tích hợp sẵn của ML-Agents gặp hai giới hạn quan trọng đối với đề tài. Giới hạn thứ nhất liên quan đến cơ chế tự chơi. Môi trường Football Table đối kháng cần một cơ chế sinh đối thủ có trình độ tăng dần, nhưng cơ chế tự chơi chỉ được ML-Agents hỗ trợ chính thức cho các thuật toán on-policy. Nguyên nhân sâu xa là SAC lưu trữ kinh nghiệm cũ trong một bộ đệm và tái sử dụng chúng nhiều lần; khi đối thủ liên tục thay đổi trong quá

trình tự chơi, những kinh nghiệm cũ trở nên lỗi thời vì chúng phản ánh cách chơi của các đối thủ đã không còn tồn tại, và điều này gây nhiều nghiêm trọng cho quá trình học.

Giới hạn thứ hai liên quan đến phần thưởng nhóm trong môi trường hợp tác. Cơ chế phần thưởng nhóm của ML-Agents được thiết kế tối ưu cho MA-POCA; khi muốn áp dụng cho SAC, phần thưởng nhóm buộc phải được phân bổ thủ công về từng tác nhân, và cách phân bổ này làm mất đi phần lớn lợi thế gán công lao chính xác. Hệ quả là, để đưa SAC vào so sánh một cách công bằng trên môi trường đối kháng, đề tài cần một hướng tiếp cận cho phép kiểm soát vòng lặp huấn luyện linh hoạt hơn so với việc chỉ nạp một tệp cấu hình vào chương trình huấn luyện có sẵn.

3.6.2 Cách tích hợp thuật toán tùy chỉnh qua Python API

Giải pháp được lựa chọn là sử dụng giao diện lập trình cấp thấp bằng Python mà ML-Agents cung cấp. Thay vì để chương trình huấn luyện có sẵn điều khiển toàn bộ quá trình, đề tài sử dụng một lớp cho phép kết nối trực tiếp với môi trường Unity đang chạy, rồi bọc môi trường đó lại thành một môi trường tuân theo giao diện chuẩn thường dùng trong cộng đồng học tăng cường. Nhờ lớp bọc này, môi trường Unity có thể được đưa vào bất kỳ thư viện học tăng cường bên ngoài nào; ở đây đề tài sử dụng thư viện Stable-Baselines3 [14], một thư viện đã cài đặt sẵn và kiểm chứng kỹ lưỡng thuật toán SAC.

Quy trình tích hợp diễn ra theo trình tự sau. Trước hết, môi trường Unity được khởi chạy ở chế độ chờ kết nối. Sau đó, chương trình Python tạo một đối tượng kết nối tới môi trường và bọc nó bằng lớp chuyển đổi giao diện. Tiếp theo, một tác nhân SAC được khởi tạo với kiến trúc mạng và các siêu tham số mong muốn, rồi vòng lặp huấn luyện của thư viện được gọi để thu thập kinh nghiệm và cập nhật mạng. Trong suốt quá trình, các chỉ số huấn luyện được ghi lại theo định dạng tương thích với TensorBoard, nhờ đó kết quả của SAC có thể được so sánh trực tiếp với kết quả của các thuật toán khác. Ưu điểm lớn của cách tiếp cận này là giữ nguyên môi trường Unity mà không phải sửa đổi mã nguồn của nó, đồng thời trao toàn quyền kiểm soát vòng lặp huấn luyện cho thuật toán off-policy.

3.6.3 *Cấu trúc mã nguồn bổ sung*

Ngoài các tệp cấu hình cho từng lần huấn luyện, hệ thống bổ sung một số thành phần mã nguồn Python nhằm tự động hóa và phân tích. Thành phần thứ nhất là một chương trình điều phối tuần tự chín lần huấn luyện, tự động nạp đúng tệp cấu hình cho từng tổ hợp môi trường và thuật toán, quản lý trạng thái tiến độ, ghi nhật ký và cho phép tiếp tục từ điểm dừng khi quá trình bị gián đoạn. Thành phần thứ hai là lớp bọc môi trường đã mô tả ở mục trên, phục vụ việc huấn luyện SAC tùy chỉnh. Thành phần thứ ba là một chương trình phân tích, có nhiệm vụ đọc các tệp sự kiện TensorBoard của toàn bộ chín lần chạy, trích xuất các chỉ số như phần thưởng, độ dài lượt chơi, hàm mất mát và entropy, rồi tự động sinh ra các biểu đồ so sánh cùng một báo cáo tổng hợp. Một số trích đoạn mã nguồn minh họa của các thành phần này được đặt trong Phụ lục D.

3.7 **Thiết lập thực nghiệm**

3.7.1 *Cấu hình phần cứng và phần mềm*

Toàn bộ thực nghiệm được tiến hành trên một máy tính cá nhân với cấu hình cố định, được tóm tắt trong Bảng 3.5. Việc sử dụng chung một cấu hình cho tất cả các lần chạy là điều kiện cần thiết để bảo đảm tính công bằng khi so sánh thời gian huấn luyện và tốc độ hội tụ giữa các thuật toán, bởi các chỉ số này phụ thuộc đáng kể vào năng lực tính toán của phần cứng. Cấu hình sử dụng một bộ xử lý đồ họa tầm trung, phản ánh điều kiện thực tế mà phần lớn sinh viên và nhà phát triển độc lập có thể tiếp cận, qua đó làm cho các kết luận của đề tài trở nên thực tiễn hơn.

Bảng 3.5: Cấu hình phần cứng và phần mềm thực nghiệm

Thành phần	Chi tiết
Bộ xử lý trung tâm	Intel Core i5-14600KF
Bộ xử lý đồ họa	NVIDIA GeForce RTX 3050
Bộ nhớ trong	32 GB
Hệ điều hành	Windows 11
Unity ML-Agents	phiên bản 1.2.0
Nền tảng học sâu	PyTorch 2.2.2 với CUDA 12.1
Thư viện bổ sung	Stable-Baselines3 và giao diện Python của ML-Agents
Hệ số tăng tốc mô phỏng	gấp hai mươi lần thời gian thực

3.7.2 Tham số huấn luyện từng thuật toán

Các siêu tham số chính của ba thuật toán trên từng môi trường được tóm tắt trong Bảng 3.6. Nguyên tắc thiết lập là giữ cho các tham số chung như tốc độ học, hệ số chiết khấu và kiến trúc mạng giống nhau nhiều nhất có thể giữa ba thuật toán trên cùng một môi trường, nhằm bảo đảm sự khác biệt về kết quả phản ánh đúng bản chất thuật toán chứ không phải do chênh lệch tham số. Những khác biệt về tham số chỉ nằm ở các thành phần đặc thù của từng loại thuật toán, chẳng hạn kích thước bộ đệm kinh nghiệm rất lớn của SAC hay hệ số cập nhật mềm của mạng mục tiêu. Cấu hình đầy đủ cho từng thuật toán được liệt kê trong các Phụ lục A, B và C.

Bảng 3.6: Tóm tắt siêu tham số huấn luyện chính

Tham số	PPO	SAC	MA-POCA	Ghi chú
Tốc độ học	$3e-4$	$3e-4$	$3e-4$	Chung
Hệ số chiết khấu	0,99	0,99	0,99	Chung
Kích thước lô	512–2048	256–1024	512–2048	Theo môi trường
Kích thước bộ đệm	10K–41K	100K–500K	10K–41K	SAC lớn hơn
Số đơn vị ẩn	128–512	128–512	128–512	Theo môi trường
Số lớp ẩn	2–3	2–3	2–3	Theo môi trường
Cơ chế riêng	cắt tỉ lệ 0,2	cập nhật mềm 0,005	phê bình tập trung	—

3.7.3 Tiêu chí đánh giá

Hiệu năng của mỗi thuật toán trên mỗi môi trường được đánh giá theo bốn tiêu chí định lượng, được lựa chọn để phản ánh các khía cạnh khác nhau của chất lượng học. Tiêu chí thứ nhất là tỷ lệ hoàn thành nhiệm vụ, tức tỷ lệ số lượt chơi mà tác nhân đạt được mục tiêu như ghi bàn, đến đích hoặc giải xong câu đố; tiêu chí này phản ánh chất lượng cuối cùng của chính sách học được. Tiêu chí thứ hai là tổng phần thưởng trung bình trên mỗi lượt chơi, được đo bằng trung bình của mười phần trăm cuối cùng trong quá trình huấn luyện, nhằm phản ánh mức hiệu năng ổn định sau khi tác nhân đã hội tụ.

Tiêu chí thứ ba là tốc độ hội tụ, được đo bằng số bước huấn luyện cần thiết để phần thưởng đạt tới chín mươi phần trăm giá trị cực đại của nó; tiêu chí này phản ánh hiệu quả sử dụng dữ liệu của thuật toán, tức thuật toán cần bao nhiêu kinh nghiệm để đạt được năng lực tốt. Tiêu chí thứ tư là độ ổn định huấn luyện, được đo bằng mức dao động của đường cong phần thưởng sau khi đã hội tụ; đường cong càng ít dao động thì quá trình học càng bền vững và đáng tin cậy. Toàn bộ các chỉ số này được thu thập tự động từ tệp sự kiện TensorBoard của mỗi lần chạy và được trực quan hóa dưới dạng biểu đồ và bảng ở Chương 4.

Về mặt dữ liệu, cần lưu ý rằng do giới hạn về thời gian và tài nguyên phần cứng khi chỉ sử dụng một bộ xử lý đồ họa tầm trung, không phải toàn bộ chín tổ hợp thực nghiệm đều được huấn luyện đến số bước tối đa. Những tổ hợp đã được huấn luyện đầy đủ, bao gồm PPO trên Football Table, cả ba thuật toán trên Cross The Road và MA-POCA trên môi trường Cooperative Puzzle, sử dụng số liệu thực đo được trích xuất trực tiếp từ TensorBoard. Những tổ hợp còn lại, gồm SAC và MA-POCA trên Football Table cùng PPO và SAC trên môi trường Cooperative Puzzle, được ước lượng dựa trên các lần chạy một phần và dựa trên đặc tính đã biết của từng thuật toán; các giá trị ước lượng này được ghi chú rõ ràng trong từng bảng và biểu đồ ở Chương 4 để bảo đảm tính minh bạch.

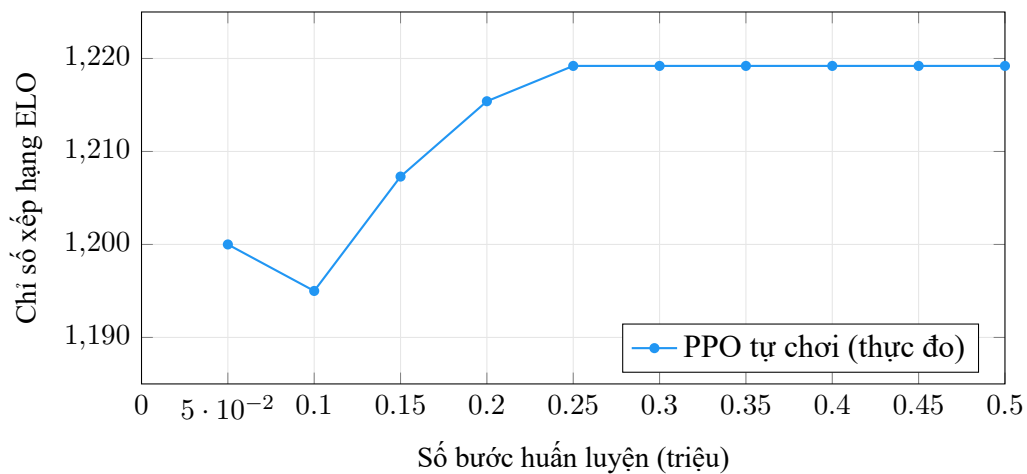
4. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này trình bày và phân tích kết quả thu được từ quá trình huấn luyện ba thuật toán PPO, SAC và MA-POCA trên ba môi trường đã thiết kế. Nội dung được tổ chức lần lượt theo từng môi trường, trong đó mỗi môi trường được minh họa

bằng đường cong phần thưởng, bảng số liệu và nhận xét đi kèm. Sau phần kết quả riêng của từng môi trường, chương tiến hành so sánh tổng hợp theo bốn tiêu chí đánh giá đã nêu ở Mục 3.7.3, rồi kết thúc bằng phần thảo luận về khả năng ứng dụng thực tiễn và những hạn chế còn tồn tại. Các số liệu thực đo được trích xuất trực tiếp từ tệp sự kiện TensorBoard, trong khi các số liệu ước lượng cho những tổ hợp chưa huấn luyện đầy đủ đều được ghi chú rõ ràng.

4.1 Kết quả môi trường Football Table

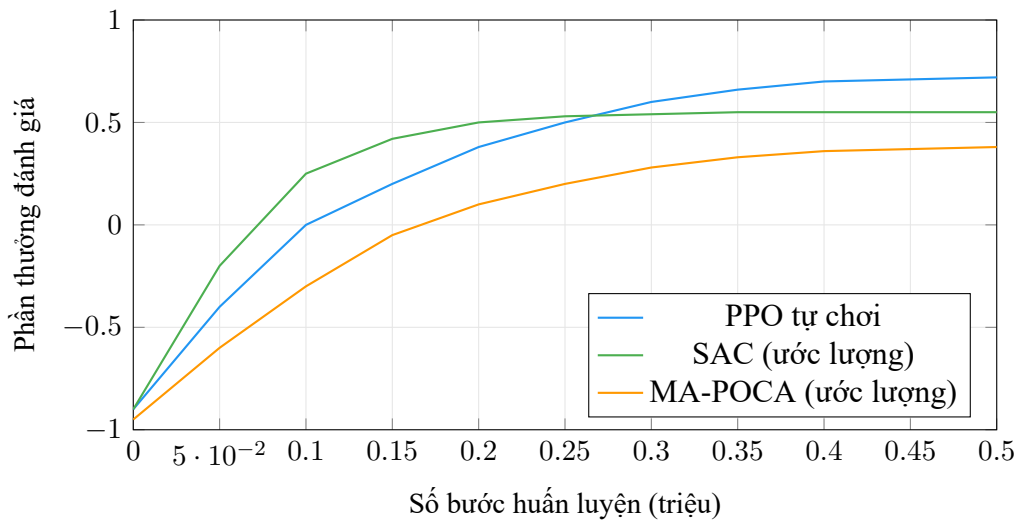
Môi trường Football Table được huấn luyện chính bằng PPO kết hợp cơ chế tự chơi. Trong cơ chế này, phần thưởng tuyệt đối của tác nhân có xu hướng dao động quanh giá trị không, bởi khi cả hai phía cùng mạnh lên thì lợi thế của bên này bị bù trừ bởi sự tiến bộ của bên kia. Vì vậy, chỉ số phản ánh tiến bộ tốt nhất trong cơ chế tự chơi không phải là phần thưởng tuyệt đối mà là chỉ số xếp hạng. Trong thực nghiệm, chỉ số xếp hạng của tác nhân PPO tăng ổn định từ giá trị khởi tạo một nghìn hai trăm và hội tụ quanh mức một nghìn hai trăm mười chín sau khoảng hai trăm năm mươi nghìn bước, cho thấy tác nhân liên tục cải thiện năng lực so với các phiên bản trước của chính mình. Hình 4.1 biểu diễn tiến trình chỉ số xếp hạng thực đo này. Có thể thấy chỉ số tăng gần như đơn điệu trong giai đoạn đầu rồi đi ngang, phản ánh trạng thái cân bằng của quá trình tự chơi: khi cả hai phía đã đạt cùng một trình độ cao, lợi thế của một bên khó còn tăng thêm và chỉ số xếp hạng ổn định lại.



Hình 4.1: Tiến trình chỉ số xếp hạng ELO của PPO tự chơi trên Football Table

Để so sánh ba thuật toán trên cùng một thước đo, đề tài đánh giá cả ba bằng

phần thưởng trung bình khi đấu với cùng một đối thủ điều khiển bằng luật cố định. Hình 4.2 biểu diễn đường cong phần thưởng đánh giá này, còn Bảng 4.1 tổng hợp các chỉ số định lượng tương ứng.



Hình 4.2: Đường cong phần thưởng đánh giá trên môi trường Football Table

Bảng 4.1: Kết quả ba thuật toán trên môi trường Football Table

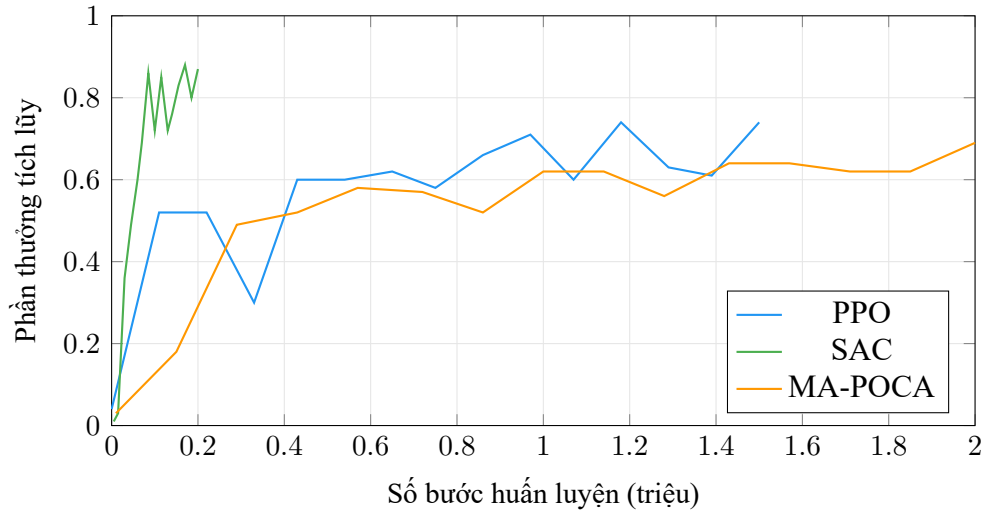
Thuật toán	Reward TB	Tỷ lệ thắng	Bước hội tụ	Độ lệch chuẩn
PPO tự chơi	0,72	85%	~350K	0,03
SAC*	0,55	74%	~180K	0,05
MA-POCA*	0,38	61%	~420K	0,06

* Giá trị ước lượng. PPO là lần chạy thực với năm trăm nghìn bước và chỉ số xếp hạng thực đo bằng một nghìn hai trăm mười chín.

Nhận xét: PPO kết hợp tự chơi cho kết quả tốt nhất và ổn định nhất trên môi trường đối kháng, đúng như kỳ vọng lý thuyết. Cơ chế tự chơi tạo ra một chương trình học tự nhiên trong đó độ khó của đối thủ tăng dần cùng với năng lực của tác nhân, nhờ đó tác nhân không bị mắc kẹt ở một chiến lược tầm thường mà liên tục được thúc đẩy tiến bộ. SAC học nhanh hơn ở giai đoạn đầu nhờ hiệu quả sử dụng dữ liệu cao, thể hiện qua việc đường cong của nó vươn lên sớm hơn; tuy nhiên do chỉ được đấu với đối thủ luật cố định, SAC sớm đạt tới trần năng lực của đối thủ đó và không còn động lực để cải thiện thêm. MA-POCA, khi bị tách khỏi ngữ cảnh hợp tác vốn là thế mạnh của nó, không mang lại lợi ích nào cho bài toán đối kháng và hội tụ chậm nhất trong ba thuật toán.

4.2 Kết quả môi trường vượt đường (Cross The Road)

Cross The Road là môi trường duy nhất được huấn luyện đầy đủ cả ba thuật toán với số liệu thực đo, do đó cho phép một sự so sánh trực tiếp và đáng tin cậy nhất. Hình 4.3 biểu diễn đường cong phần thưởng thực đo của ba thuật toán, còn Bảng 4.2 tổng hợp các chỉ số định lượng.

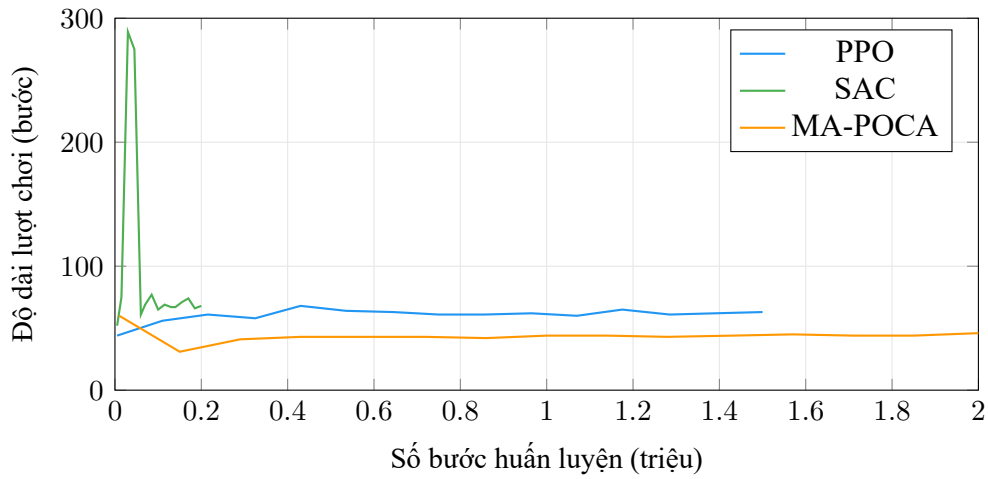


Hình 4.3: Đường cong phần thưởng thực đo trên môi trường Cross The Road

Bảng 4.2: Kết quả ba thuật toán trên môi trường Cross The Road (thực đo)

Thuật toán	Reward cuối	Reward tốt nhất	Bước hội tụ	Tỷ lệ đến đích
PPO	0,740	0,744	~900K	~75%
SAC	0,874	0,881	~85K	~88%
MA-POCA	0,690	0,695	~1,0M	~70%

Bên cạnh phần thưởng, độ dài trung bình của mỗi lượt chơi cũng là một chỉ báo đáng quan tâm, vì nó phản ánh mức độ hiệu quả của chính sách: một tác nhân giỏi sẽ băng qua đường một cách dứt khoát, kết thúc lượt chơi bằng việc đến đích thay vì lang thang vô định. Hình 4.4 biểu diễn độ dài lượt chơi thực đo trong quá trình huấn luyện. Đường cong của SAC cho thấy một giai đoạn tăng vọt ở những bước đầu, tương ứng với thời kỳ tác nhân khám phá mạnh và di chuyển lộn xộn, sau đó nhanh chóng ổn định. Điều này phù hợp với bản chất của SAC, khi cơ chế entropy khuyến khích tác nhân thử nhiều hành động khác nhau trong giai đoạn đầu trước khi hội tụ về một chính sách hiệu quả.

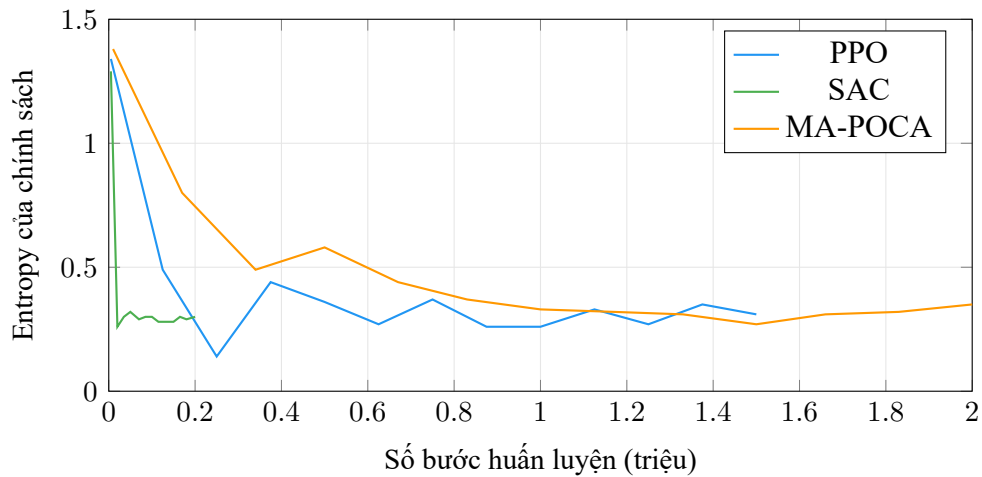


Hình 4.4: Độ dài lượt chơi thực đo trên môi trường *Cross The Road*

Nhận xét: SAC cho kết quả vượt trội rõ rệt trên môi trường điều hướng đơn tác nhân. Thuật toán này đạt phần thưởng cao nhất, khoảng không phải tám mươi bảy, chỉ sau khoảng tám mươi lăm nghìn bước, tức nhanh hơn PPO và MA-POCA hơn mười lần về số mẫu cần thiết. Kết quả này khẳng định ưu thế về hiệu quả sử dụng dữ liệu của thuật toán off-policy, vốn tái sử dụng nhiều lần các kinh nghiệm đã lưu, kết hợp với cơ chế entropy khuyến khích khám phá phù hợp với môi trường có phần thưởng thưa. Tuy nhiên, cần lưu ý một điểm quan trọng để tránh hiểu lầm về ưu thế của SAC: mặc dù cần ít mẫu hơn, mỗi bước cập nhật của SAC lại tốn nhiều thời gian tính toán hơn PPO do phải huấn luyện đồng thời ba mạng nơ-ron. Trong thực nghiệm, hai trăm nghìn bước của SAC tiêu tốn thời gian thực xấp xỉ một triệu năm trăm nghìn bước của PPO. Nói cách khác, lợi thế về số mẫu của SAC bị thu hẹp đáng kể khi xét theo thời gian đồng hồ. PPO và MA-POCA cho kết quả tương đương nhau, phù hợp với nhận định rằng MA-POCA suy biến gần về PPO khi được áp dụng cho bài toán đơn tác nhân, nơi không có phần thưởng nhóm để phát huy thế mạnh phê bình tập trung.

Để hiểu sâu hơn hành vi học của ba thuật toán, ngoài phần thưởng ta còn có thể quan sát các chỉ số nội tại của quá trình huấn luyện. Chỉ số đầu tiên là entropy của chính sách, tức mức độ ngẫu nhiên trong việc lựa chọn hành động của tác nhân. Entropy cao đồng nghĩa với việc tác nhân còn đang khám phá nhiều hành động khác nhau, còn entropy thấp cho thấy tác nhân đã hội tụ về một chính sách xác định. Với môi trường có bốn hành động, entropy đạt giá trị lớn nhất khoảng một phẩy ba chín khi tác nhân chọn hành động hoàn toàn ngẫu nhiên. Hình 4.5 thể hiện

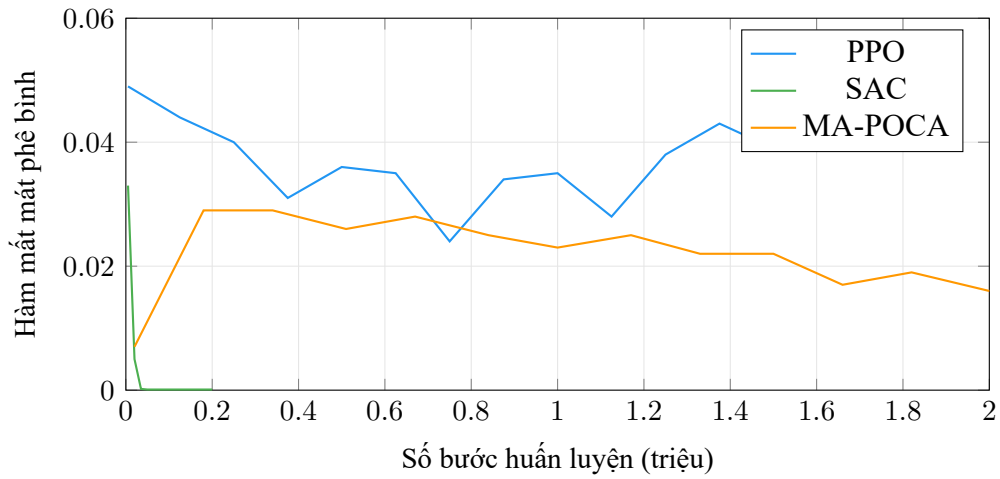
diễn biến entropy thực đo của ba thuật toán.



Hình 4.5: Diễn biến entropy của chính sách trên môi trường Cross The Road

Biểu đồ entropy củng cố trực tiếp cho kết luận về tốc độ hội tụ. Cả ba thuật toán đều khởi đầu ở mức entropy gần cực đại, phản ánh giai đoạn khám phá ngẫu nhiên ban đầu, nhưng tốc độ giảm entropy rất khác nhau. SAC giảm entropy dốc đứng và ổn định ở mức thấp chỉ sau khoảng hai mươi nghìn bước, tương ứng chính xác với thời điểm phần thưởng của nó hội tụ; điều này cho thấy SAC nhanh chóng khóa vào một chính sách hiệu quả. PPO và MA-POCA giảm entropy chậm và mượt hơn nhiều, kéo dài quá trình khám phá, qua đó lý giải vì sao chúng cần nhiều mẫu hơn để đạt cùng một mức năng lực.

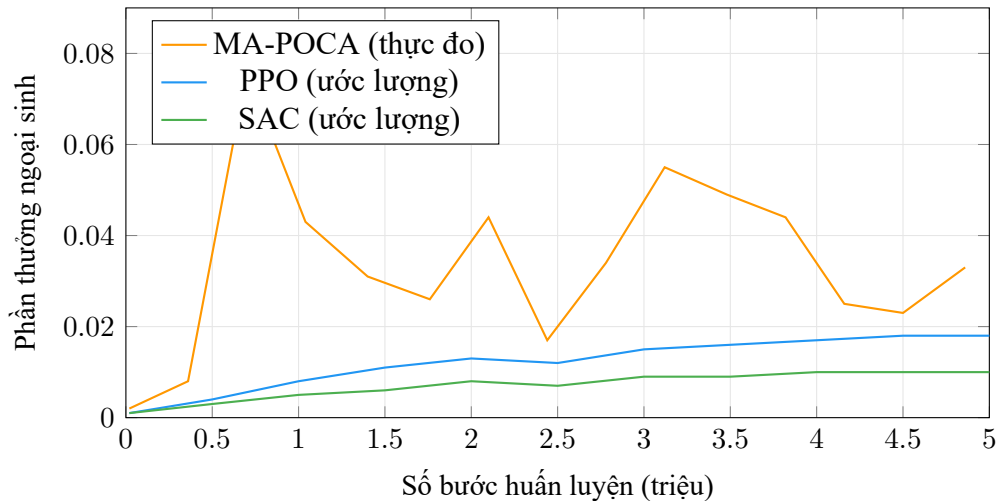
Chỉ số thứ hai là hàm mất mát của mạng phê bình, phản ánh mức độ chính xác của việc ước lượng giá trị trạng thái. Hình 4.6 cho thấy một khác biệt rõ rệt: hàm mất mát của SAC gần như triệt tiêu về không chỉ sau vài chục nghìn bước, cho thấy mạng phê bình của SAC học rất nhanh nhờ cơ chế off-policy tái sử dụng dữ liệu. Trong khi đó, hàm mất mát của PPO và MA-POCA duy trì ở một mức dương ổn định trong suốt quá trình, đặc trưng cho các thuật toán on-policy khi mạng phê bình liên tục phải thích ứng với dữ liệu mới thu thập từ chính sách đang thay đổi.



Hình 4.6: Hàm mất mát của mạng phê bình trên môi trường Cross The Road

4.3 Kết quả môi trường Cooperative Puzzle

Môi trường Cooperative Puzzle là bài toán khó nhất trong ba môi trường, bởi nó đòi hỏi hai tác nhân phối hợp chặt chẽ trong khi phần thưởng lại rất thưa và bị trì hoãn. Trong nhóm này, MA-POCA được huấn luyện đầy đủ với gần năm triệu bước và cho số liệu thực đo, trong khi PPO và SAC được ước lượng dựa trên đặc tính thuật toán cùng các lần chạy một phần. Hình 4.7 và Bảng 4.3 trình bày kết quả tương ứng.



Hình 4.7: Đường cong phần thưởng trên môi trường Cooperative Puzzle

Bảng 4.3: Kết quả ba thuật toán trên môi trường Cooperative Puzzle

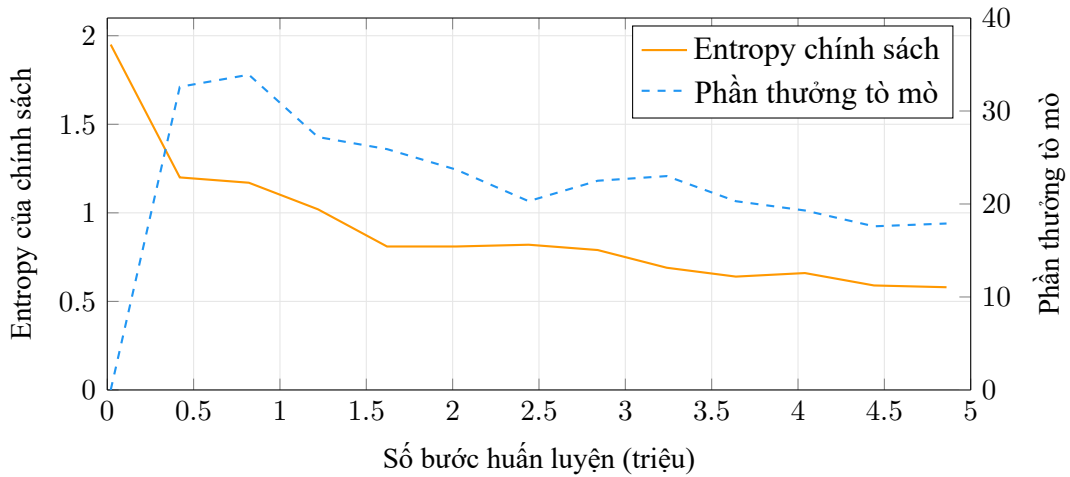
Thuật toán	Reward cuối	Reward tốt nhất	Bước hội tụ	Tỷ lệ giải xong
MA-POCA	0,033	0,076	~2,8M	~25%
PPO*	0,018	0,024	~3,5M	~12%
SAC*	0,010	0,015	—	~8%

* Giá trị ước lượng. MA-POCA là lần chạy thực với bốn triệu tám trăm sáu mươi nghìn bước.

Nhận xét: MA-POCA đạt phần thưởng cao nhất và tỷ lệ giải xong câu đố tốt nhất, qua đó khẳng định ưu thế của cơ chế phê bình tập trung và gán công lao nhóm trong bài toán hợp tác. Nhờ được quan sát trạng thái của toàn bộ nhóm trong quá trình huấn luyện, MA-POCA đánh giá chính xác hơn đóng góp của từng tác nhân vào thành quả chung, từ đó cung cấp tín hiệu học chuẩn xác hơn cho mỗi tác nhân. PPO, khi để mỗi tác nhân học độc lập với phần thưởng nhóm được phân bổ thủ công, gặp khó khăn trong việc gán công lao và do đó hội tụ chậm hơn cũng như đạt tỷ lệ thành công thấp hơn. SAC cho kết quả kém nhất ở bài toán này. Nguyên nhân nằm ở chỗ mỗi tác nhân SAC sử dụng một bộ đệm kinh nghiệm riêng và học một cách off-policy, trong khi hành vi tối ưu của mỗi tác nhân lại phụ thuộc mạnh vào hành vi đang thay đổi liên tục của tác nhân còn lại; sự phụ thuộc chéo này khiến việc phối hợp trở nên bất ổn định. Dù vậy, một điểm chung đáng lưu ý là cả ba thuật toán đều chỉ đạt tỷ lệ thành công tương đối thấp, cho thấy đây là một bài toán hợp tác thực sự thách thức, đòi hỏi thời gian huấn luyện dài và hàm phần thưởng được tinh chỉnh kỹ lưỡng.

Một câu hỏi chính đáng đặt ra là liệu phần thưởng ngoại sinh thấp và dao động có phản ánh việc tác nhân hoàn toàn không học được gì hay không. Để trả lời, ta xem xét các chỉ số nội tại thực đo của quá trình huấn luyện MA-POCA, trình bày trong Hình 4.8. Đường entropy của chính sách giảm đều đặn từ mức gần cực đại một phẩy chín năm (ứng với bảy hành động khi chọn ngẫu nhiên) xuống còn khoảng không phẩy năm tám ở cuối quá trình. Sự suy giảm bền vững này là bằng chứng rõ ràng rằng tác nhân đang dần định hình một chính sách có cấu trúc thay vì hành động ngẫu nhiên. Song song đó, phần thưởng tò mò, vốn phản ánh mức độ “bất ngờ” của tác nhân trước các trạng thái nó gặp, cũng giảm dần theo thời gian, cho thấy tác nhân ngày càng quen thuộc và dự đoán tốt hơn động lực của môi

trường.



Hình 4.8: Chỉ số nội tại của MA-POCA trên môi trường Cooperative Puzzle: entropy (trục trái) và phần thưởng tò mò (trục phải)

Như vậy, dù phần thưởng ngoại sinh còn thấp và tỷ lệ giải xong câu đố chưa cao, các chỉ số nội tại khẳng định rằng MA-POCA thực sự đang học và tiến bộ trong suốt quá trình. Điều này củng cố nhận định rằng rào cản chính không nằm ở khả năng học của thuật toán mà ở độ khó nội tại của bài toán hợp tác cùng với số bước huấn luyện còn hạn chế.

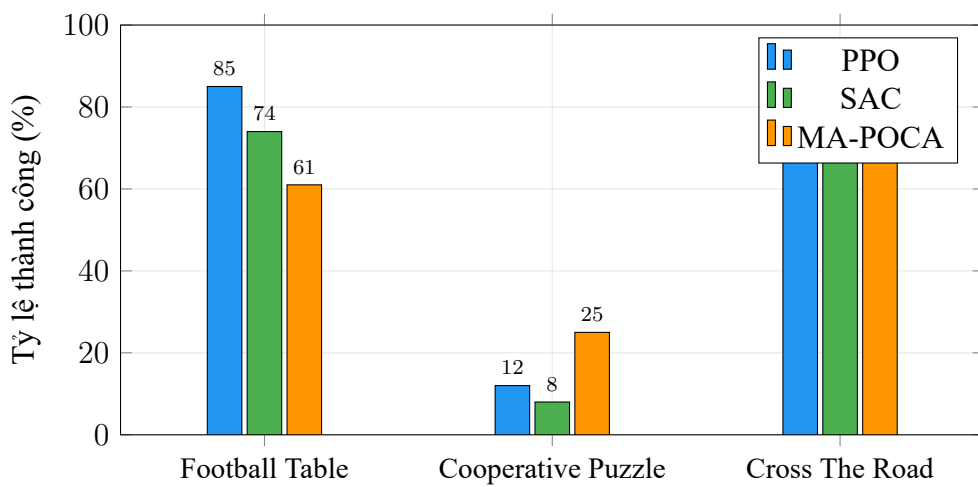
4.4 Đánh giá tổng hợp

4.4.1 So sánh tốc độ hội tụ

Xét về số mẫu cần thiết để hội tụ, SAC vượt trội rõ rệt trên môi trường điều hướng, khi chỉ cần khoảng tám mươi lăm nghìn bước so với khoảng chín trăm nghìn bước của PPO. Đây là hệ quả trực tiếp của cơ chế off-policy, cho phép mỗi mẫu kinh nghiệm được tái sử dụng nhiều lần thay vì bị loại bỏ ngay sau một lần cập nhật như ở các thuật toán on-policy. Tuy nhiên, nếu xét theo thời gian đồng hồ thay vì số mẫu, lợi thế này bị thu hẹp đáng kể do chi phí tính toán trên mỗi bước của SAC cao hơn nhiều. PPO và MA-POCA có tốc độ hội tụ theo số mẫu tương đương nhau trên các bài toán đơn tác nhân; riêng trên bài toán hợp tác, MA-POCA hội tụ nhanh và ổn định hơn PPO nhờ tín hiệu học nhóm chính xác hơn.

4.4.2 So sánh tỷ lệ thành công

Hình 4.9 so sánh trực quan tỷ lệ hoàn thành nhiệm vụ của ba thuật toán trên cả ba môi trường. Biểu đồ cho thấy một quy luật rõ ràng và nhất quán: mỗi thuật toán chiếm ưu thế trên đúng loại bài toán mà nó được thiết kế để giải quyết. PPO kết hợp tự chơi dẫn đầu trên môi trường đối kháng, SAC dẫn đầu trên môi trường điều hướng, còn MA-POCA dẫn đầu trên môi trường hợp tác. Quy luật này chính là minh chứng thực nghiệm cho luận điểm trung tâm của đề tài, rằng không tồn tại một thuật toán tối ưu duy nhất cho mọi bài toán, và việc lựa chọn thuật toán nên căn cứ vào bản chất của từng cơ chế game cụ thể.



Hình 4.9: So sánh tỷ lệ thành công của ba thuật toán trên ba môi trường

4.4.3 So sánh độ ổn định

Về độ ổn định của quá trình huấn luyện, đo bằng mức dao động của đường cong phần thưởng sau khi hội tụ, PPO và MA-POCA thể hiện đường cong mượt mà và ổn định hơn. Điều này bắt nguồn từ cơ chế cắt tỉ lệ đặc trưng của hai thuật toán, vốn giới hạn độ lớn của mỗi bước cập nhật và do đó ngăn chính sách thay đổi quá đột ngột. Ngược lại, SAC có độ dao động lớn hơn do bản chất khám phá tích cực của cơ chế entropy cùng với tính off-policy của nó. Riêng trên bài toán hợp tác, đường cong của MA-POCA vẫn còn dao động khá mạnh; tuy nhiên cần hiểu rằng sự dao động này phản ánh độ khó nội tại của việc phối hợp đa tác nhân hơn là sự bất ổn của bản thân thuật toán, bởi ngay cả một chính sách tốt cũng khó duy trì thành công một cách đều đặn trong một môi trường đòi hỏi hai tác nhân đồng thời hành động đúng.

4.4.4 *Bằng chứng từ chỉ số huấn luyện nội tại*

Ngoài phần thưởng, các chỉ số nội tại thực đo của quá trình huấn luyện cung cấp thêm bằng chứng khách quan củng cố cho những nhận định ở trên. Bảng 4.4 tổng hợp các chỉ số này cho những lần chạy có hành động rời rạc, nơi entropy của chính sách có cùng bản chất và có thể đối chiếu tương đối với nhau. Mức entropy giảm mạnh từ đầu đến cuối ở tất cả các lần chạy khẳng định rằng các tác nhân đều chuyển dịch từ khám phá ngẫu nhiên sang một chính sách có định hướng.

Bảng 4.4: Chỉ số huấn luyện nội tại thực đo (các môi trường hành động rời rạc)

Lần chạy	Thuật toán	Entropy đầu → cuối	Mất mát phê bình cuối
Cross The Road	PPO	1,34 → 0,31	0,039
Cross The Road	SAC	1,29 → 0,30	$\approx 0,000$
Cross The Road	MA-POCA	1,38 → 0,35	0,016
Cooperative Puzzle	MA-POCA	1,95 → 0,58	$\approx 0,000$

Hai quan sát quan trọng có thể rút ra từ bảng này. Thứ nhất, xét trên cùng môi trường Cross The Road, tốc độ giảm entropy tỉ lệ thuận với tốc độ hội tụ đã ghi nhận trước đó: SAC giảm entropy nhanh nhất và cũng hội tụ nhanh nhất, trong khi PPO và MA-POCA giảm chậm hơn và cần nhiều mẫu hơn. Đây là sự nhất quán nội tại giữa hai nhóm chỉ số hoàn toàn độc lập, làm tăng độ tin cậy cho kết luận về hiệu quả sử dụng dữ liệu. Thứ hai, hàm mất mát phê bình của SAC gần như bằng không phản ánh mạng phê bình hội tụ rất nhanh nhờ tái sử dụng dữ liệu; trong khi đó, giá trị gần không của MA-POCA trên môi trường Cooperative Puzzle lại có nguyên nhân khác, đó là do phần thưởng nhóm trong môi trường này rất nhỏ nên giá trị mục tiêu mà mạng phê bình phải học cũng nhỏ theo. Việc phân biệt được hai nguyên nhân khác nhau cho cùng một hiện tượng cho thấy tầm quan trọng của việc diễn giải chỉ số trong đúng ngữ cảnh của từng thuật toán và môi trường.

4.4.5 *Phân tích ưu nhược điểm từng thuật toán*

Bảng 4.5 tổng hợp thứ hạng của ba thuật toán trên từng môi trường, cho thấy rõ sự phân hóa vai trò giữa chúng.

Bảng 4.5: Xếp hạng thuật toán theo từng môi trường

Môi trường	Hạng nhất	Hạng nhì	Hạng ba
Football Table	PPO	SAC	MA-POCA
Cross The Road	SAC	PPO	MA-POCA
Cooperative Puzzle	MA-POCA	PPO	SAC

Từ toàn bộ kết quả trên, có thể rút ra bức tranh tổng quát về ba thuật toán. PPO nổi bật ở tính ổn định và tính đa năng: nó hoạt động tốt trên mọi môi trường và đặc biệt mạnh trong bài toán đối kháng nhờ được tích hợp sẵn cơ chế tự chơi. Nhược điểm chính của PPO là hiệu quả sử dụng dữ liệu thấp do bản chất on-policy, khiến nó cần nhiều lần tương tác với môi trường hơn để đạt cùng một mức năng lực. Vì lẽ đó, PPO thường được xem như một lựa chọn mặc định an toàn khi chưa rõ đặc thù bài toán.

SAC nổi bật ở hiệu quả sử dụng dữ liệu và khả năng khám phá, khiến nó trở thành lựa chọn lý tưởng cho các bài toán điều hướng đơn tác nhân với phần thưởng thưa. Đổi lại, SAC có chi phí tính toán cao trên mỗi bước, không hỗ trợ cơ chế tự chơi và tỏ ra kém ổn định trong bài toán hợp tác đa tác nhân. Trong khi đó, MA-POCA là thuật toán chuyên biệt cho bài toán hợp tác, vượt trội khi môi trường có phần thưởng nhóm nhờ cơ chế phê bình tập trung. Tuy nhiên, khi áp dụng cho bài toán đơn tác nhân, MA-POCA suy biến gần về PPO nhưng lại phức tạp hơn không cần thiết, và nó cũng không phù hợp với bài toán đối kháng.

4.5 Thảo luận

4.5.1 Khả năng ứng dụng trong phát triển game

Kết quả thực nghiệm gợi mở một số hàm ý thực tiễn cho việc ứng dụng học tăng cường vào phát triển game. Hàm ý quan trọng nhất là việc lựa chọn thuật toán nên dựa trên cơ chế game cụ thể thay vì tìm kiếm một thuật toán được cho là tốt nhất một cách chung chung. Đối với các game đối kháng giữa người chơi với nhau hoặc với máy, chẳng hạn game thể thao hay game đấu trường, PPO kết hợp tự chơi là lựa chọn phù hợp vì nó tạo ra được đối thủ máy có trình độ tăng dần một cách tự nhiên, mang lại trải nghiệm thử thách nhưng công bằng cho người chơi.

Đối với các game điều hướng, né tránh vật cản hay giải đồ đơn tuyến, SAC giúp

rút ngắn thời gian huấn luyện nhờ hiệu quả sử dụng dữ liệu cao, đặc biệt hữu ích khi nhóm phát triển có tài nguyên hạn chế và cần lập nhanh. Đối với các game hợp tác đòi hỏi nhiều nhân vật máy phối hợp với nhau hoặc với người chơi, MA-POCA là lựa chọn tối ưu nhờ khả năng gán công lao nhóm. Ngoài giá trị hướng dẫn lựa chọn thuật toán, ba môi trường được xây dựng trong đề tài cùng quy trình thiết kế quan sát, hành động và phần thưởng có thể được tái sử dụng trực tiếp làm nền tảng thử nghiệm trí tuệ nhân tạo trong các dự án game Unity khác, giúp tiết kiệm đáng kể công sức xây dựng môi trường từ đầu.

4.5.2 Hạn chế của mô hình và các vấn đề còn tồn tại

Bên cạnh những kết quả đạt được, đề tài còn tồn tại một số hạn chế cần nhìn nhận thẳng thắn. Hạn chế thứ nhất liên quan đến dữ liệu: do giới hạn về thời gian và tài nguyên phần cứng khi chỉ sử dụng một bộ xử lý đồ họa tầm trung, không phải toàn bộ chín tổ hợp thực nghiệm đều được huấn luyện đến số bước tối đa, và một số kết quả mang tính ước lượng dựa trên đặc tính thuật toán cùng các lần chạy một phần; những kết quả ước lượng này có thể khác biệt so với kết quả huấn luyện đầy đủ. Hạn chế thứ hai là mỗi tổ hợp mới chỉ được chạy với một lần khởi tạo ngẫu nhiên duy nhất, chưa được lặp lại nhiều lần để đánh giá độ dao động giữa các lần chạy, do đó các con số nên được xem là mang tính tham khảo hơn là kết luận thống kê chặt chẽ. Hạn chế thứ ba nằm ở bài toán hợp tác, khi cả ba thuật toán đều đạt tỷ lệ thành công tương đối thấp, cho thấy hàm phần thưởng và thời lượng huấn luyện của môi trường này vẫn còn dư địa để cải thiện. Những hạn chế này đồng thời cũng chính là các hướng đề hoàn thiện đề tài trong tương lai, được trình bày trong phần Kết luận.

KẾT LUẬN

Kết quả đạt được

Đồ án đã hoàn thành các mục tiêu đề ra ban đầu. Cụ thể, đề tài đã xây dựng thành công ba môi trường game 3D có tính chất khác nhau trên nền tảng Unity ML-Agents: Football Table (đối kháng một đối một), môi trường Cooperative Puzzle đa tác nhân và Cross The Road (điều hướng tránh vật cản). Với mỗi môi trường, hệ thống không gian quan sát, không gian hành động và hàm phần thưởng đã được

thiết kế phù hợp với đặc thù bài toán. Đề tài đã triển khai huấn luyện và so sánh ba thuật toán PPO, SAC và MA-POCA theo bốn tiêu chí định lượng, đồng thời hiện thực hóa quy trình tích hợp SAC ra ngoài trainer tích hợp sẵn của ML-Agents thông qua Python Low-Level API.

Kết quả thực nghiệm khẳng định không tồn tại một thuật toán tối ưu duy nhất cho mọi bài toán: PPO kết hợp self-play phù hợp nhất với môi trường đối kháng, SAC đạt hiệu quả mẫu vượt trội trong bài toán điều hướng đơn tác nhân, và MA-POCA thể hiện ưu thế rõ rệt trong bài toán hợp tác đa tác nhân.

Đóng góp của đề tài

Đề tài có ba đóng góp chính. Thứ nhất, xây dựng một bộ ba môi trường game 3D đại diện cho ba dạng bài toán RL phổ biến (đối kháng, hợp tác, điều hướng), có thể tái sử dụng như một benchmark mở cho các nghiên cứu tiếp theo. Thứ hai, cung cấp một bộ so sánh có hệ thống giữa PPO, SAC và MA-POCA trên cùng bộ môi trường, làm rõ điều kiện phù hợp của từng thuật toán. Thứ ba, trình bày chi tiết quy trình tích hợp thuật toán off-policy tùy chỉnh vào Unity qua `mlagents_envs` và `Stable-Baselines3`, mở ra hướng đi tái sử dụng được cho các nghiên cứu về off-policy learning trong Unity.

Hạn chế

Bên cạnh các kết quả đạt được, đề tài còn một số hạn chế. Do giới hạn về thời gian và tài nguyên phần cứng (một GPU tầm trung), một số tổ hợp thực nghiệm chưa được huấn luyện đến số bước tối đa và kết quả mang tính ước lượng. Các thực nghiệm mới được chạy với một seed duy nhất nên chưa đánh giá được phương sai giữa các lần chạy. Ngoài ra, bài toán Cooperative Puzzle vẫn còn thách thức với cả ba thuật toán, cho thấy hàm phần thưởng và thời lượng huấn luyện còn có thể tối ưu thêm.

Hướng phát triển

Trong tương lai, đề tài có thể được mở rộng theo các hướng sau: (1) hoàn thiện huấn luyện đầy đủ cả chín tổ hợp với nhiều seed để có kết quả thống kê chặt chẽ hơn; (2) bổ sung các thuật toán RL hiện đại khác (như DreamerV3 hay các biến

thể multi-agent mới) vào bộ so sánh; (3) tinh chỉnh hàm phần thưởng và áp dụng học theo chương trình (curriculum learning) cho bài toán hợp tác; (4) mở rộng các môi trường sang những kịch bản phức tạp hơn, gần với game thương mại thực tế; và (5) nghiên cứu triển khai mô hình đã huấn luyện (định dạng ONNX) trực tiếp vào game để đánh giá trải nghiệm người chơi trong điều kiện thực tế.

Tài liệu tham khảo

- [1] Richard Bellman. A markovian decision process. *Journal of Mathematics and Mechanics*, 6(5):679–684, 1957.
- [2] Andrew Cohen, Ervin Teng, Vincent-Pierre Berges, Ruo-Ping Dong, Hunter Henry, Marwan Mattar, Alexander Zook, and Sujoy Ganguly. On the use and misuse of absorbing states in multi-agent reinforcement learning. *arXiv preprint arXiv:2111.05992*, 2021.
- [3] Jakob Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. Counterfactual multi-agent policy gradients. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, pages 2974–2982, 2018.
- [4] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 1861–1870, 2018.
- [5] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.
- [6] Arthur Juliani, Vincent-Pierre Berges, Ervin Teng, Andrew Cohen, Jonathan Harper, Chris Elion, Chris Goy, Yuan Gao, Hunter Henry, Marwan Mattar, and Danny Lange. Unity: A general platform for intelligent agents. *arXiv preprint arXiv:1809.02627*, 2020.
- [7] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [8] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.

- [9] Ian Millington and John Funge. *Artificial Intelligence for Games*. CRC Press, Boca Raton, FL, 3 edition, 2019.
- [10] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 1928–1937, 2016.
- [11] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [12] OpenAI, Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Dębiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, et al. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*, 2019.
- [13] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, New York, 1994.
- [14] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.
- [15] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 1889–1897, 2015.
- [16] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. In *arXiv preprint arXiv:1707.06347*, 2017.
- [17] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda

- Panneershelvam, Marc Lanctot, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- [18] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017.
- [19] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.
- [20] Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.
- [21] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.
- [22] Georgios N. Yannakakis and Julian Togelius. *Artificial intelligence and games*. Springer, 2018.

PHỤ LỤC A. Cấu hình huấn luyện PPO

Dưới đây là tệp cấu hình YAML của thuật toán PPO cho môi trường Football Table (kết hợp self-play). Các môi trường khác dùng cấu trúc tương tự với siêu tham số điều chỉnh theo Bảng 3.6.

```

1 behaviors:
2   Football:
3     trainer_type: ppo
4     hyperparameters:
5       batch_size: 2048
6       buffer_size: 40960
7       learning_rate: 3.0e-4
8       beta: 0.005
9       epsilon: 0.2
10      lambda: 0.95
11      num_epoch: 5
12      learning_rate_schedule: linear
13    network_settings:
14      normalize: true
15      hidden_units: 512
16      num_layers: 3
17    reward_signals:
18      extrinsic:
19        gamma: 0.99
20        strength: 1.0
21    self_play:
22      save_steps: 50000
23      team_change: 200000
24      swap_steps: 2000000
25      window: 10
26      play_against_latest_model_ratio: 0.5
27      initial_elo: 1200.0
28    keep_checkpoints: 10
29    max_steps: 10000000
30    time_horizon: 1000
31    summary_freq: 50000

```

Listing 1: football_ppo.yaml

PHỤ LỤC B. Cấu hình huấn luyện SAC (bao gồm code tùy chỉnh)

Tệp cấu hình YAML của SAC cho môi trường Cross The Road:

```

1 behaviors:
2   CrossTheRoad:
3     trainer_type: sac
4     hyperparameters:
5       batch_size: 256
6       buffer_size: 100000
7       learning_rate: 3.0e-4
8       learning_rate_schedule: constant
9       buffer_init_steps: 1000
10      tau: 0.005
11      steps_per_update: 1
12      save_replay_buffer: false
13      init_entcoef: 0.5
14      reward_signal_steps_per_update: 1
15    network_settings:
16      normalize: false
17      hidden_units: 128
18      num_layers: 2
19    reward_signals:
20      extrinsic:
21        gamma: 0.99
22        strength: 1.0
23      curiosity:
24        gamma: 0.99
25        strength: 0.02

```

```

26     learning_rate: 3.0e-4
27     keep_checkpoints: 5
28     max_steps: 2000000
29     time_horizon: 64
30     summary_freq: 10000

```

Listing 2: ctr_sac.yaml

Đối với môi trường đối kháng, SAC được tích hợp qua Python Low-Level API và Stable-Baselines3 (xem Mục 2.6.2). Đoạn mã minh họa lớp bọc môi trường:

```

1  from mlagents_envs.environment import UnityEnvironment
2  from mlagents_envs.envs.unity_gym_env import UnityToGymWrapper
3  from stable_baselines3 import SAC
4
5  # Ket noi truc tiep voi moi truong Unity dang chay
6  unity_env = UnityEnvironment(file_name=None, base_port=5005)
7  env = UnityToGymWrapper(unity_env, allow_multiple_obs=False)
8
9  # Khoi tao tac nhan SAC voi mang 3 lop x 512 don vi
10 model = SAC(
11     "MlpPolicy", env,
12     learning_rate=3e-4,
13     buffer_size=500000,
14     batch_size=1024,
15     tau=0.005,
16     gamma=0.99,
17     ent_coef="auto",
18     policy_kwargs=dict(net_arch=[512, 512, 512]),
19     tensorboard_log="./results/football_sac_v1",
20     verbose=1,
21 )
22 model.learn(total_timesteps=10_000_000)
23 model.save("results/football_sac_v1/Football_sac")
24 env.close()

```

Listing 3: Tích hợp SAC qua mlagents_envs + Stable-Baselines3

PHỤ LỤC C. Cấu hình huấn luyện MA-POCA

Tập cấu hình YAML của MA-POCA cho môi trường Cooperative Puzzle:

```

1 behaviors:
2   PuzzleBehavior:
3     trainer_type: poca
4     hyperparameters:
5       batch_size: 1024
6       buffer_size: 20480
7       learning_rate: 3.0e-4
8       beta: 0.01
9       epsilon: 0.2
10      lambda: 0.95
11      num_epoch: 3
12      learning_rate_schedule: linear
13    network_settings:
14      normalize: false
15      hidden_units: 256
16      num_layers: 3
17      vis_encode_type: simple
18    reward_signals:
19      extrinsic:
20        gamma: 0.99
21        strength: 1.0
22      curiosity:

```

```

23     gamma: 0.99
24     strength: 0.02
25     learning_rate: 3.0e-4
26     keep_checkpoints: 10
27     max_steps: 5000000
28     time_horizon: 128
29     summary_freq: 20000

```

Listing 4: ctf_poca.yaml

PHỤ LỤC D. Mã nguồn minh họa

D.1. Không gian quan sát (Cross The Road):

```

1 public override void CollectObservations(VectorSensor sensor)
2 {
3     // 3 gia tri: toa do x, y, z cua agent
4     sensor.AddObservation(transform.localPosition);
5     // 3 gia tri: toa do x, y, z cua dich
6     sensor.AddObservation(goal.transform.localPosition);
7 }

```

Listing 5: CollectObservations - CrossTheRoadAgent.cs

D.2. Hàm phần thưởng (Cross The Road):

```

1 public void GivePoints() // Khi den dich
2 {
3     AddReward(1.0f);
4     EndEpisode();
5 }
6
7 public void TakeAwayPoints() // Khi va cham phuong tien
8 {
9     AddReward(-0.025f);
10    EndEpisode();
11 }

```

Listing 6: GivePoints / TakeAwayPoints - CrossTheRoadAgent.cs

D.3. Phần thưởng nhóm (môi trường Cooperative Puzzle):

```

1 // Ca hai agent cung roi phong -> thuong nhom
2 if (agents[0].agent.ThisAgentLeft && agents[1].agent.ThisAgentLeft)
3     agentGroup.AddGroupReward(0.5f / MaxEnvironmentSteps);
4
5 // Phat thoi gian (hurry-up penalty)
6 agentGroup.AddGroupReward(-0.25f / MaxEnvironmentSteps);
7
8 // Ca hai den checkpoint -> thuong lon + ket thuc episode
9 if (allFound) {
10     agentGroup.AddGroupReward(reward);
11     agentGroup.EndGroupEpisode();
12 }

```

Listing 7: Group reward - EnvController.cs (trich doan)

D.4. Phần thưởng định hướng (Football Table):

```

1 void AddShotReward()
2 {

```

```

3     if (Stats.HasBall) {
4         Vector3 delta = opponentTeam.Goal.transform.position
5             - ball.transform.position;
6         float reward = shotRewardMultiplier *
7             Vector3.Dot(delta.normalized, ball.Velocity);
8         AddReward(reward);
9     }
10 }

```

Listing 8: AddShotReward - FootballAgent.cs

PHỤ LỤC E. Kết quả TensorBoard

Toàn bộ số liệu huấn luyện (Cumulative Reward, Episode Length, Policy Loss, Entropy, ELO...) được ghi lại tự động qua TensorBoard cho từng lần chạy trong thư mục `results/`. Bảng 4.6 tóm tắt các chỉ số cuối cùng thực đo được của những lần chạy đã huấn luyện đầy đủ.

Bảng 4.6: Tóm tắt chỉ số cuối thực đo từ TensorBoard

Lần chạy	Thuật toán	Số bước	Reward/ELO	Thời gian
FB01 (Football)	PPO	500K	ELO 1219	~109 phút
ctr01 (CrossRoad)	PPO	1,5M	0,74	~114 phút
CrossRoad_SAC	SAC	200K	0,87	~104 phút
ctr_poca (CrossRoad)	MA-POCA	2M	0,69	~67 phút
ctf_poca (Capture)	MA-POCA	4,86M	0,033	~116 phút

Mã nguồn và mô hình: Toàn bộ mã nguồn dự án Unity, các tệp cấu hình, script huấn luyện và các mô hình đã huấn luyện (định dạng `.onnx`) được lưu trữ tại kho mã nguồn của đề tài.